
Logistic Regression

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Logistic Regression – Formalization
- Gradient descent solution, Regularization
- Polynomial Logistic Regression
- Multiclass logistic regression
- Summary

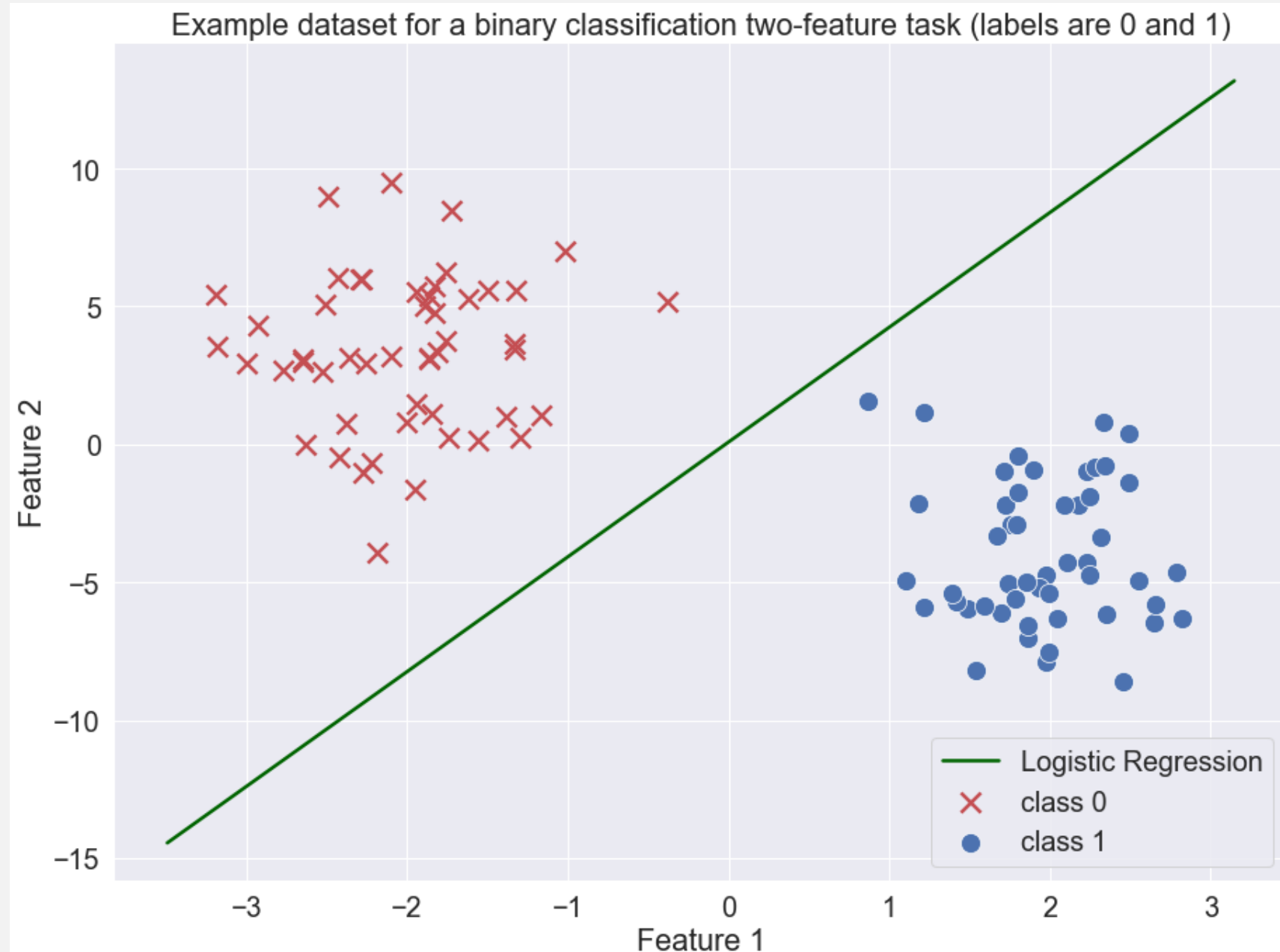
Formalization

- Consider this two-feature binary classification problem. Labels are 0 and 1.
- Notice we can plot this on a 2D plane with feature 1 and feature 2 axes. We just give a different label marker for class 0 and 1.
- There is a clear boundary between the 0 and 1 classes.
- How do we find a line boundary that separates the two classes?



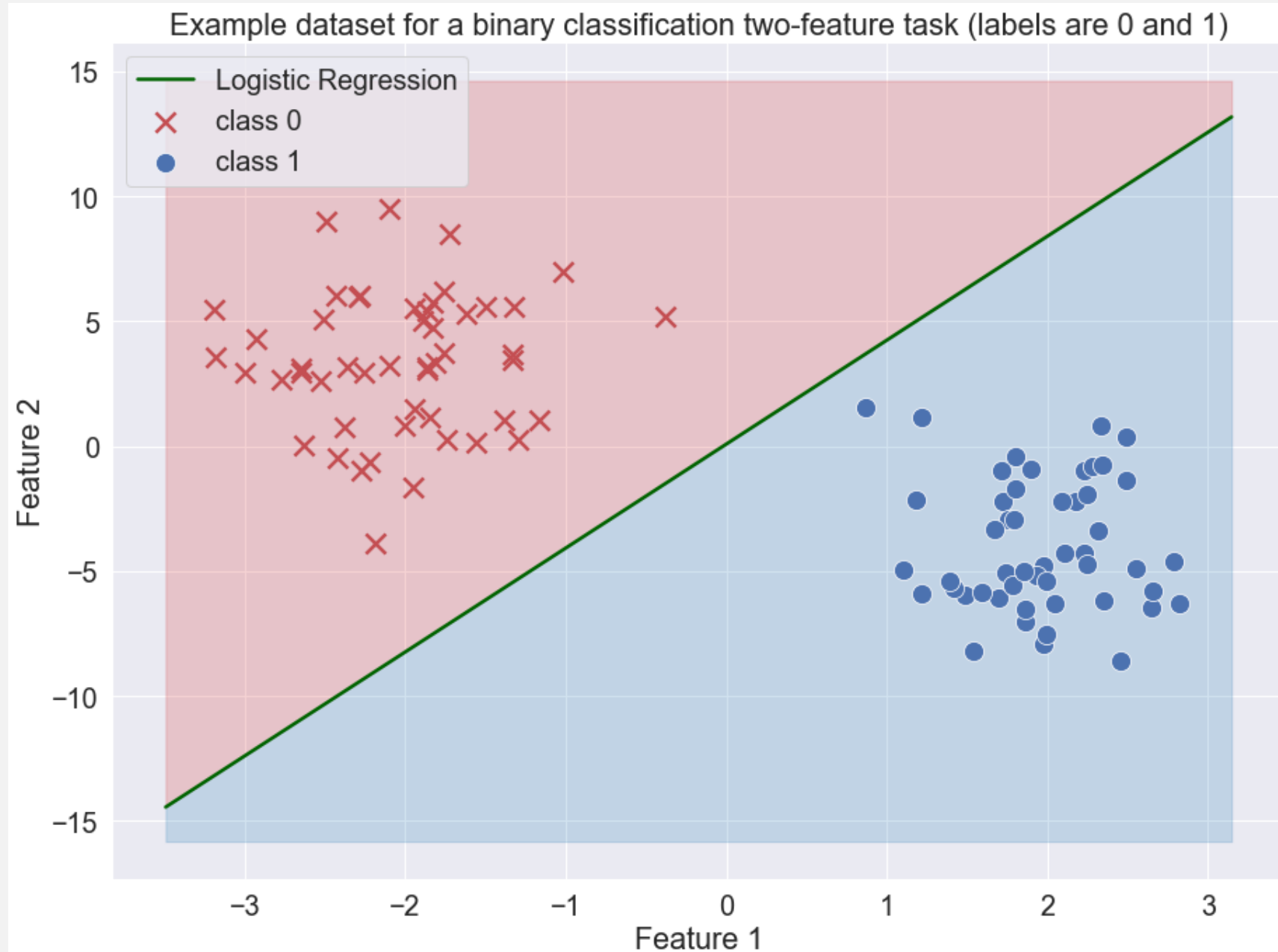
Motivation [2]

- How do we find a line boundary that separates the two classes?
- Fitting logistic regression to this data results in the green line.
- Points that lie to the left of the green line are predicted to be class 0.
- Points that lie to the right of the green line are predicted to be class 1.



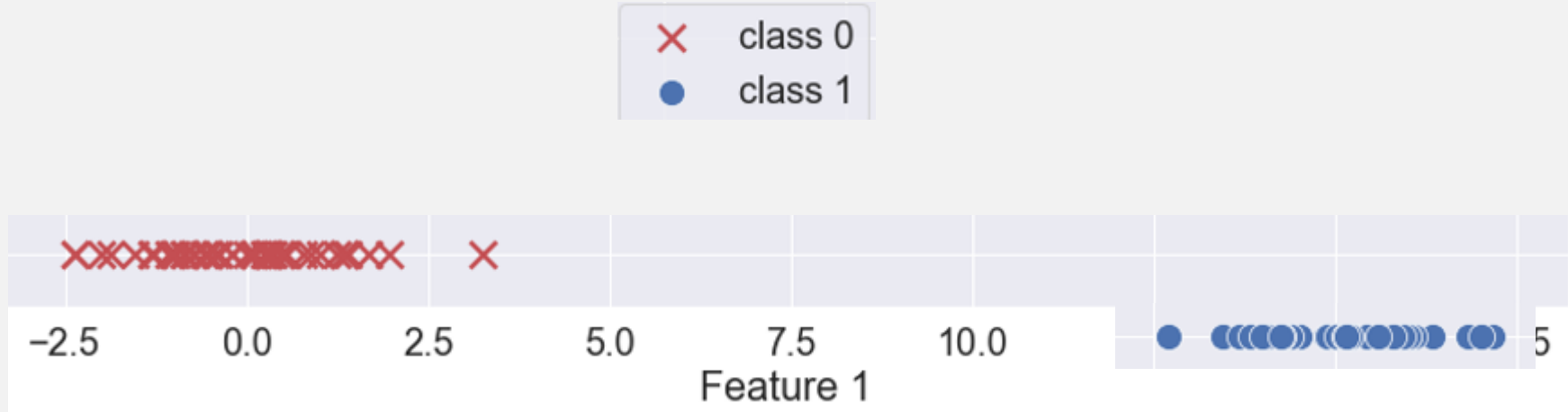
Motivation [3]

- In other words, the green line represents a **decision boundary**. This in turn defines the **decision surface**.
- Points lying in the red zone are class 0.
- Points lying in the blue zone are class 1.
- How do we find such decision lines? Use logistic regression.



Issues [1]

- Now consider this binary classification dataset with just one feature. The output labels are 0 and 1.
- What happens if we fit ordinary linear regression to this data?



Issues [2]

- Let's put the data on the 2D cartesian plane with y-axis representing label.
- What happens if we fit ordinary linear regression to this data?

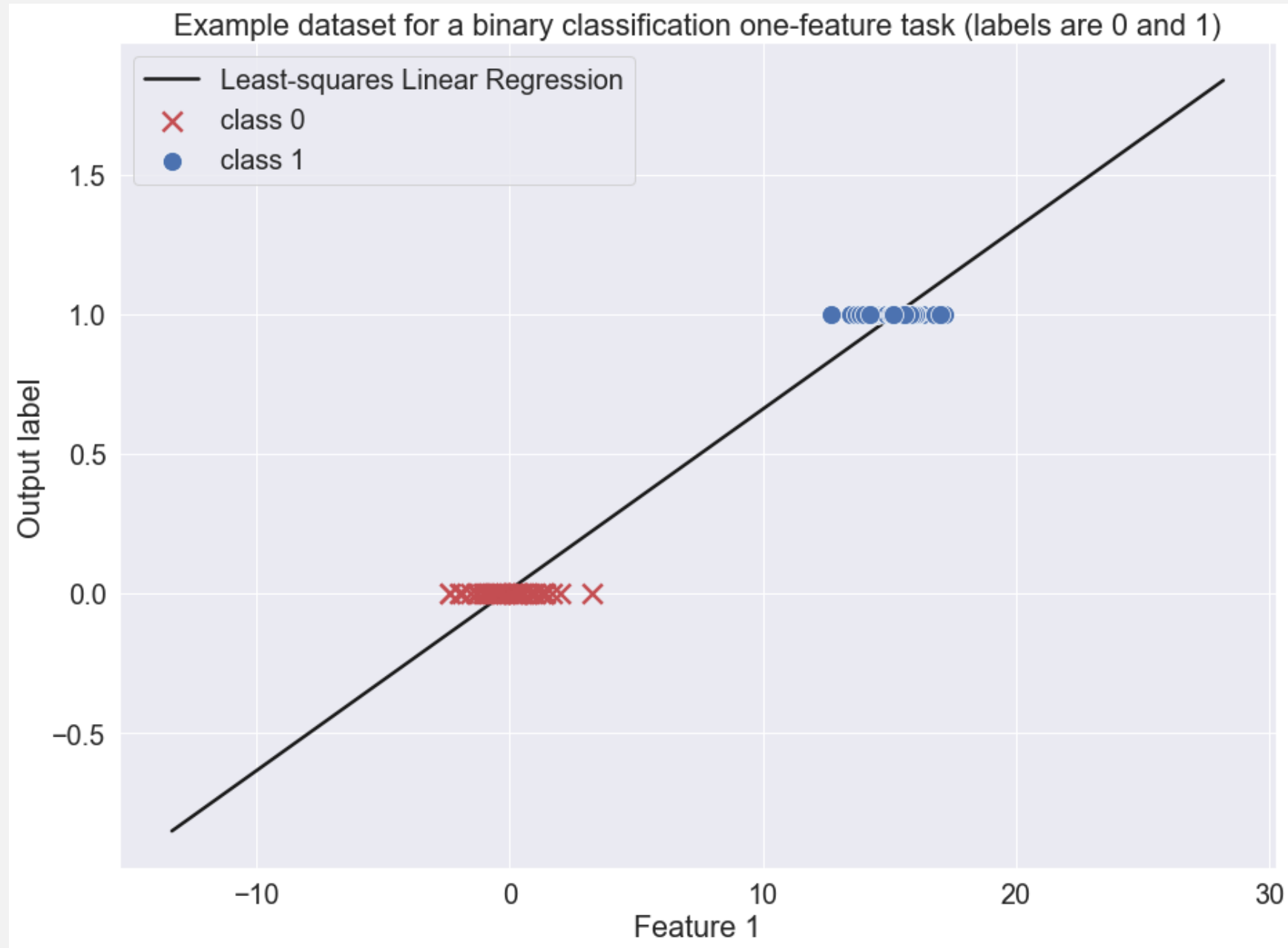


Issues [3]

- What happens if we fit ordinary linear regression to this data?

$$\hat{y}_i = x_i^T w$$

- The issue is that the linear regression line will give predictions that are above 1 and below 0.
- For very large values of Feature 1, the prediction will be very large as well, much larger than 1.
- For very small values (negative) for Feature 1, the prediction will be very negative as well, much smaller than 0.
- Can we fix this issue with a clipping function?

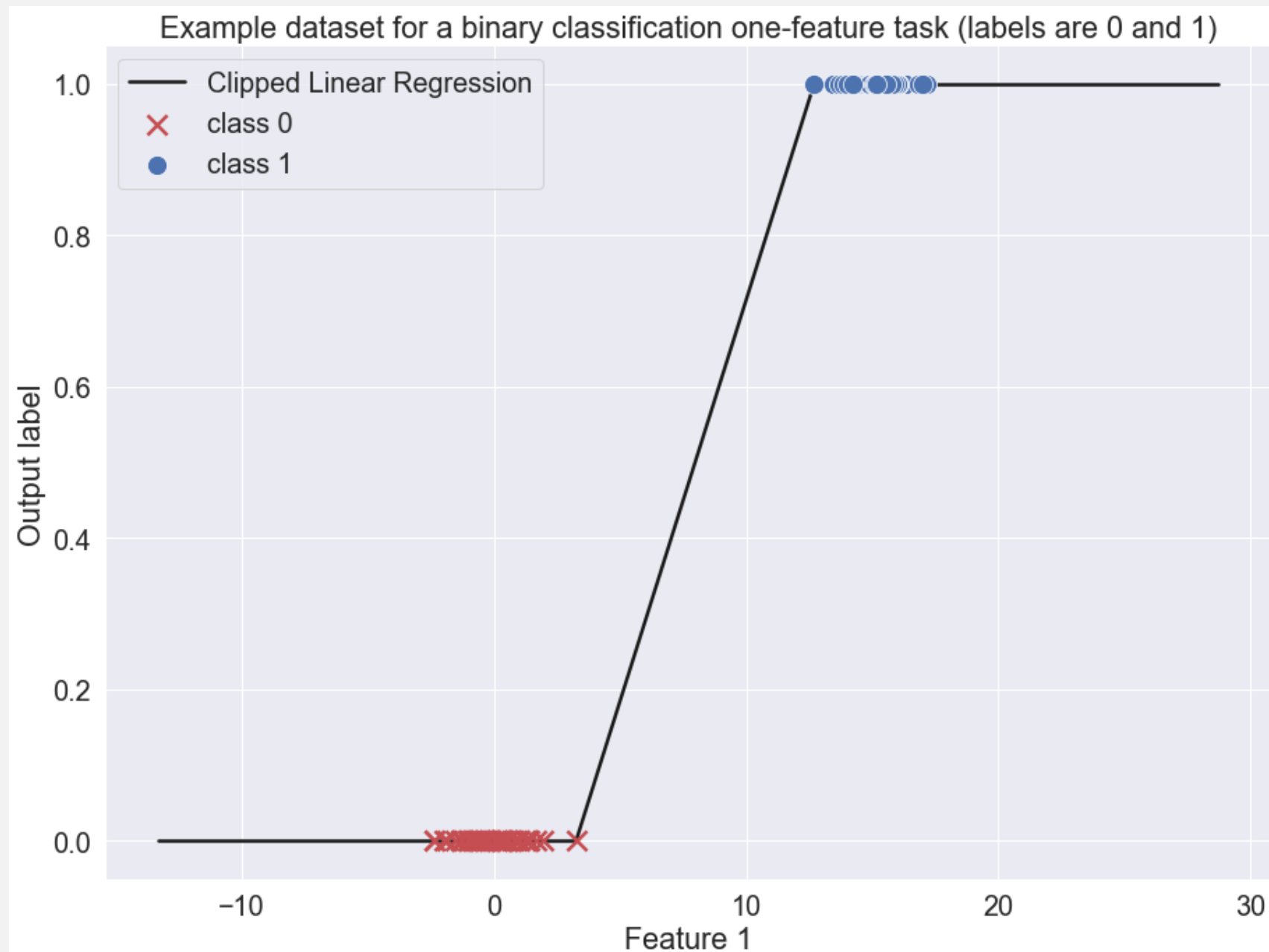


Attempt to fix the issue

- Can we fix this issue with a clipping function?

$$\hat{y}_i = \text{clip}(x_i^T w, \min=0, \max=1)$$

- This is much better. Large prediction values are clipped to 1. Small prediction values are clipped to 0.
- However, there is another issue. Recall that we compute gradients to minimize a loss. With this clipping function, computing the gradient can be problematic. We prefer *smooth* functions when differentiating.



Sigmoid (Logistic) Function

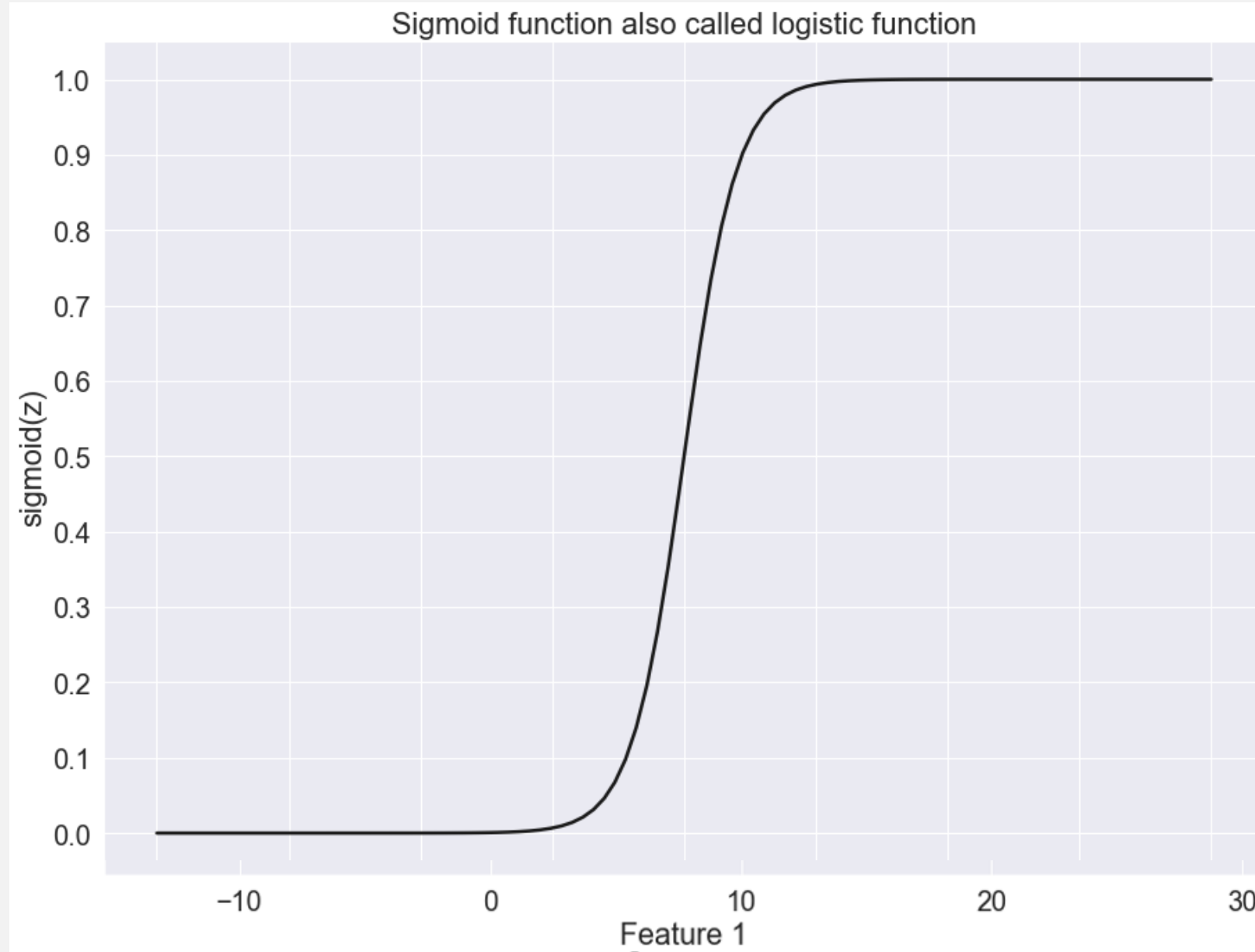
- Is there a function that mimics the clipping function, but is smooth and easily differentiable?
- Yes, the sigmoid function (also called logistic function):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma(\infty) \rightarrow 1, \sigma(-\infty) \rightarrow 0, \sigma(0) = 0.5$$

- The derivative is:

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$$

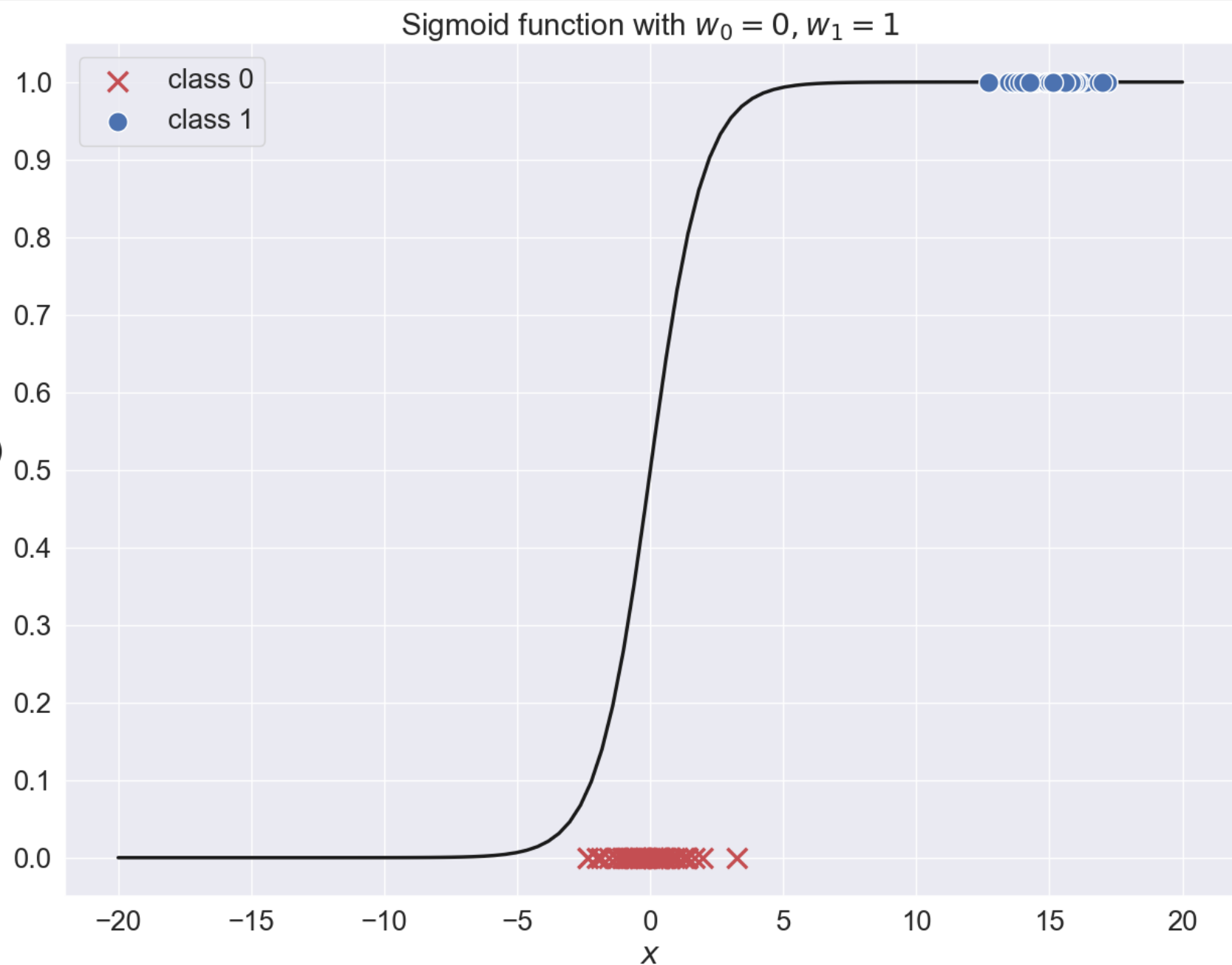


Logistic Regression 1 feature example

- So, let's try to fit the data to this sigmoid function with weights.

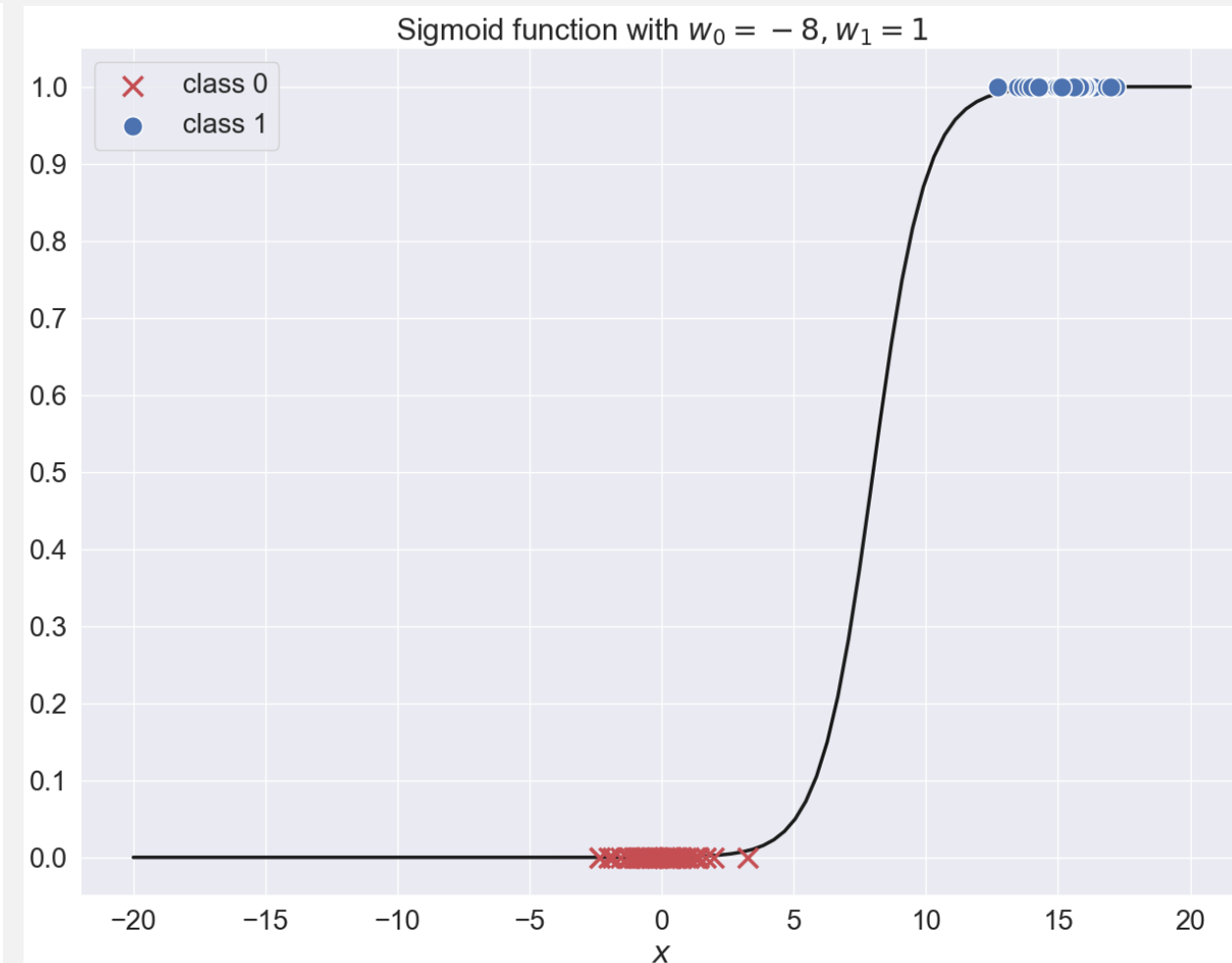
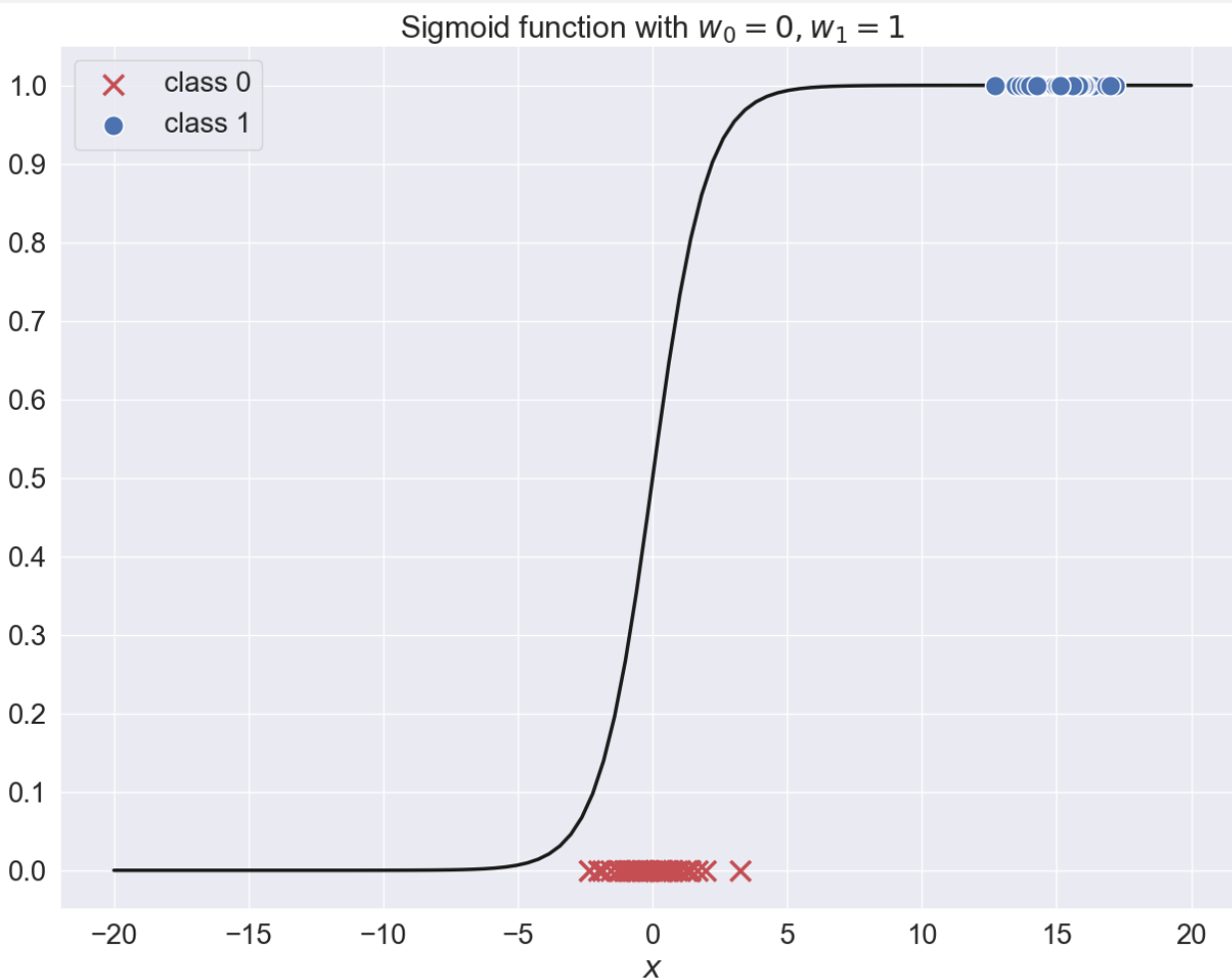
$$\hat{y} = \sigma(x^T w)$$
$$= \frac{1}{1 + e^{-x^T w}}$$

$$\hat{y} = \sigma(w_0 + w_1 x)$$



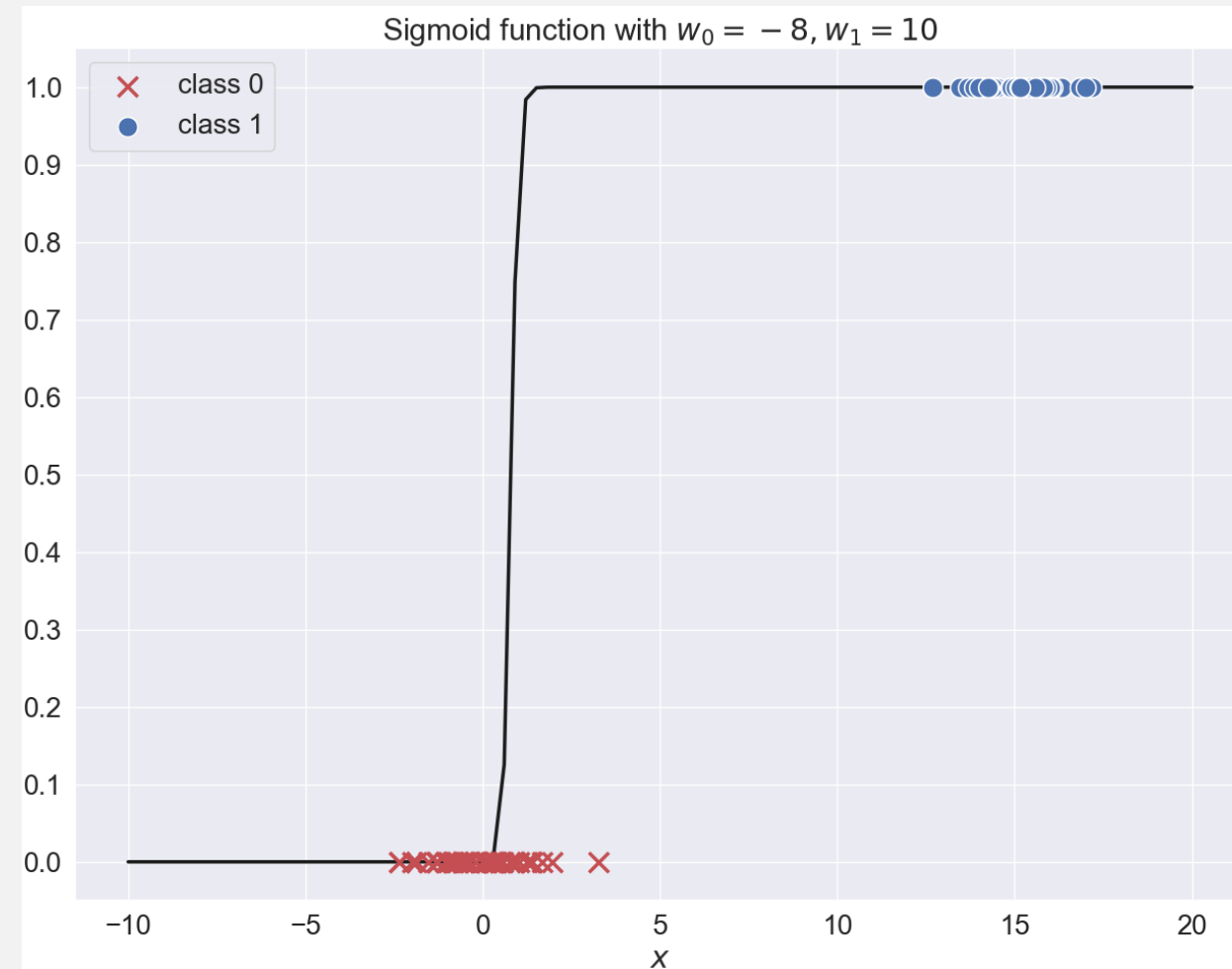
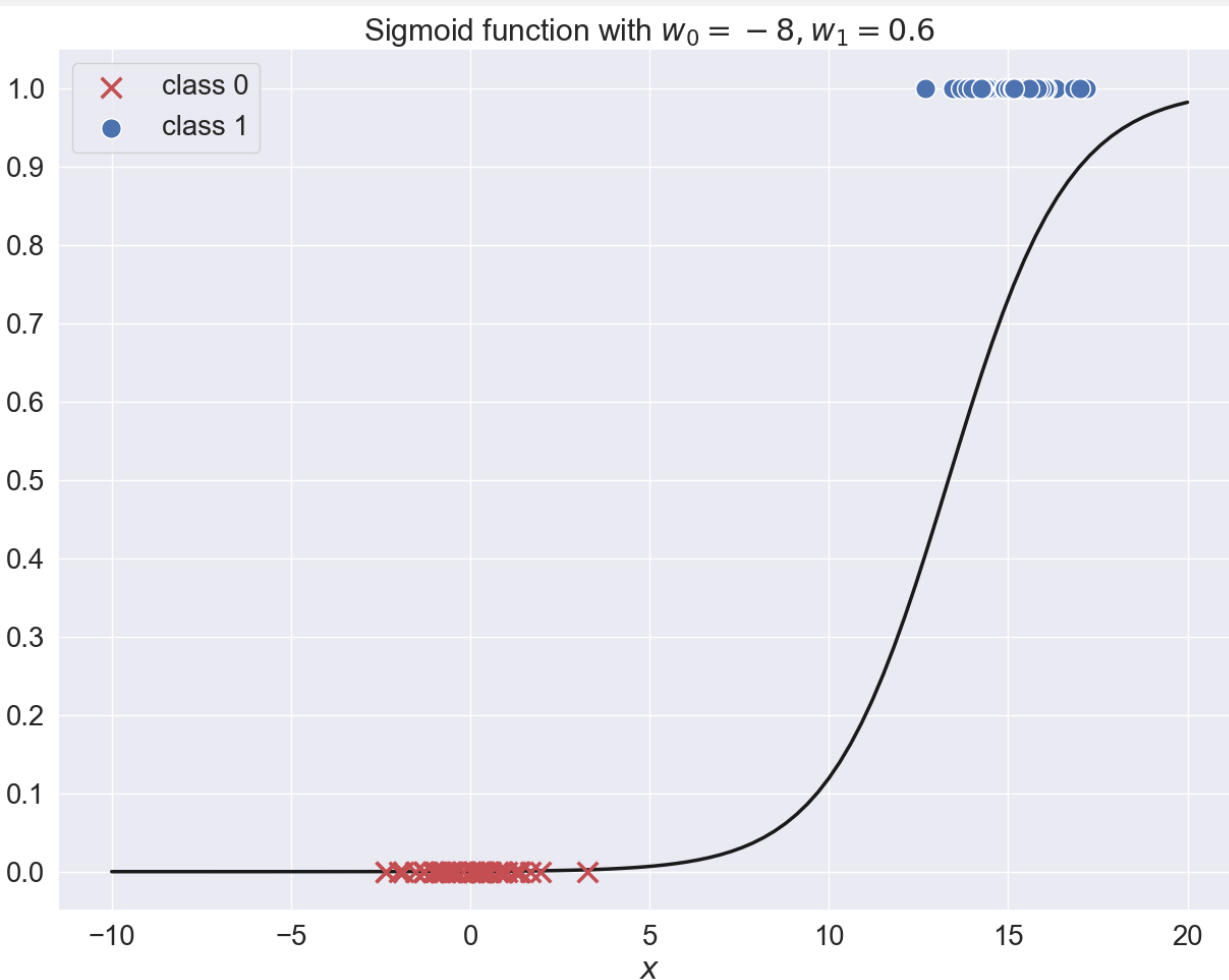
Logistic Regression 1 feature example

- Notice that as we vary the weights, the slope and positioning of the sigmoid changes. The bias weight controls the position, the rest of the weights control the slope.
- Logistic regression tries to find best slope and positioning for the sigmoid to put as many points at the correct limits.

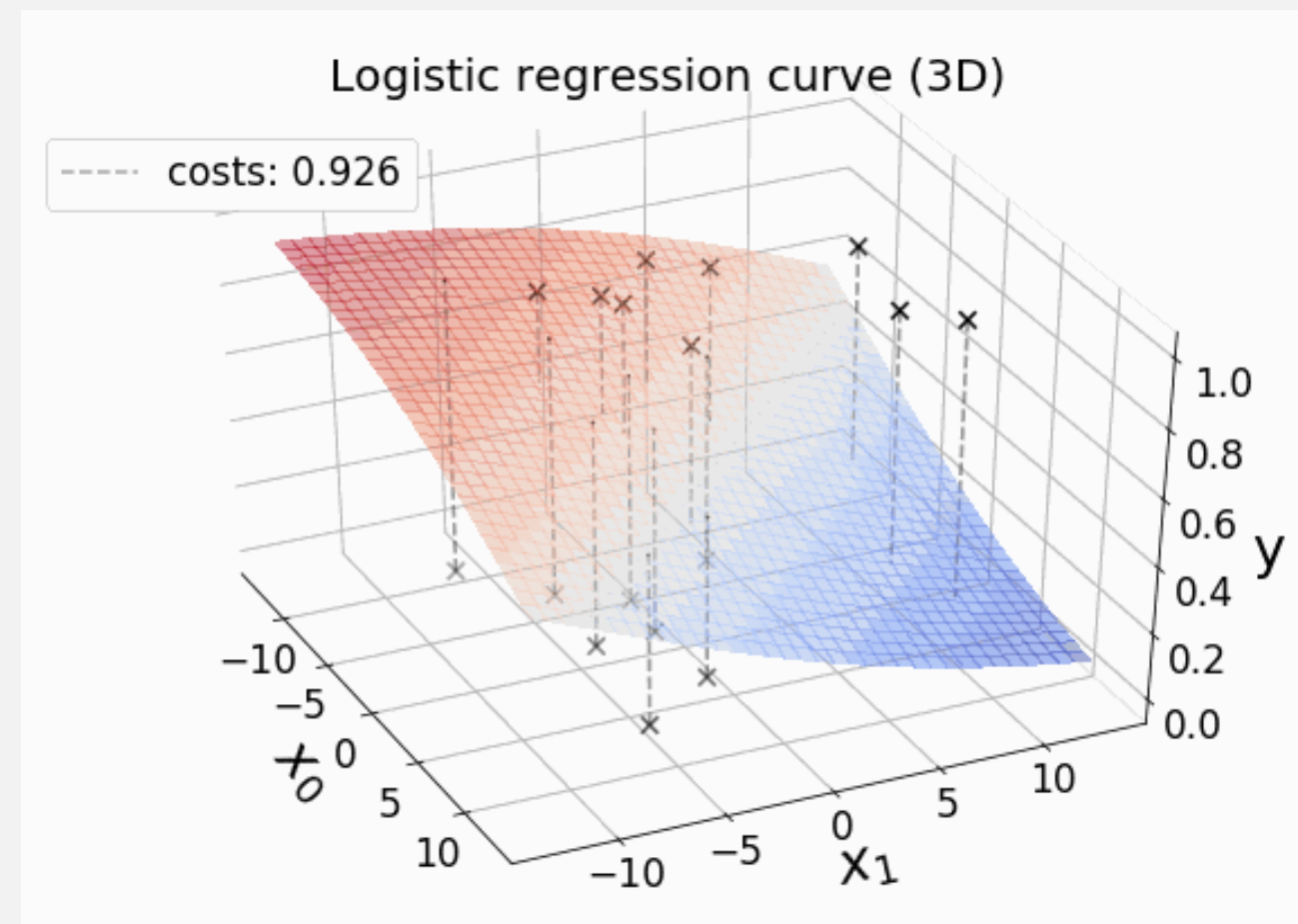
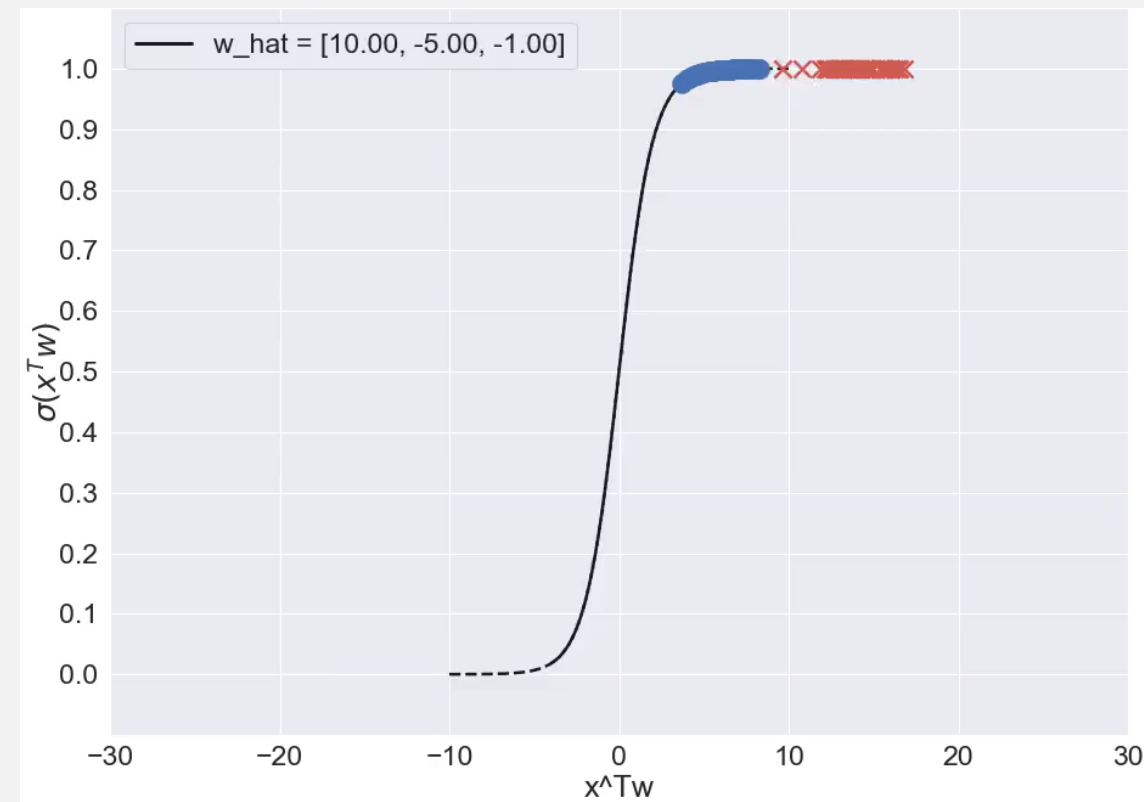


Logistic Regression 1 feature example

- Notice that as we vary the weights, the slope and positioning of the sigmoid changes. The bias weight controls the position, the rest of the weights control the slope.
- Logistic regression tries to find best slope and positioning for the sigmoid to put as many points at the correct limits.



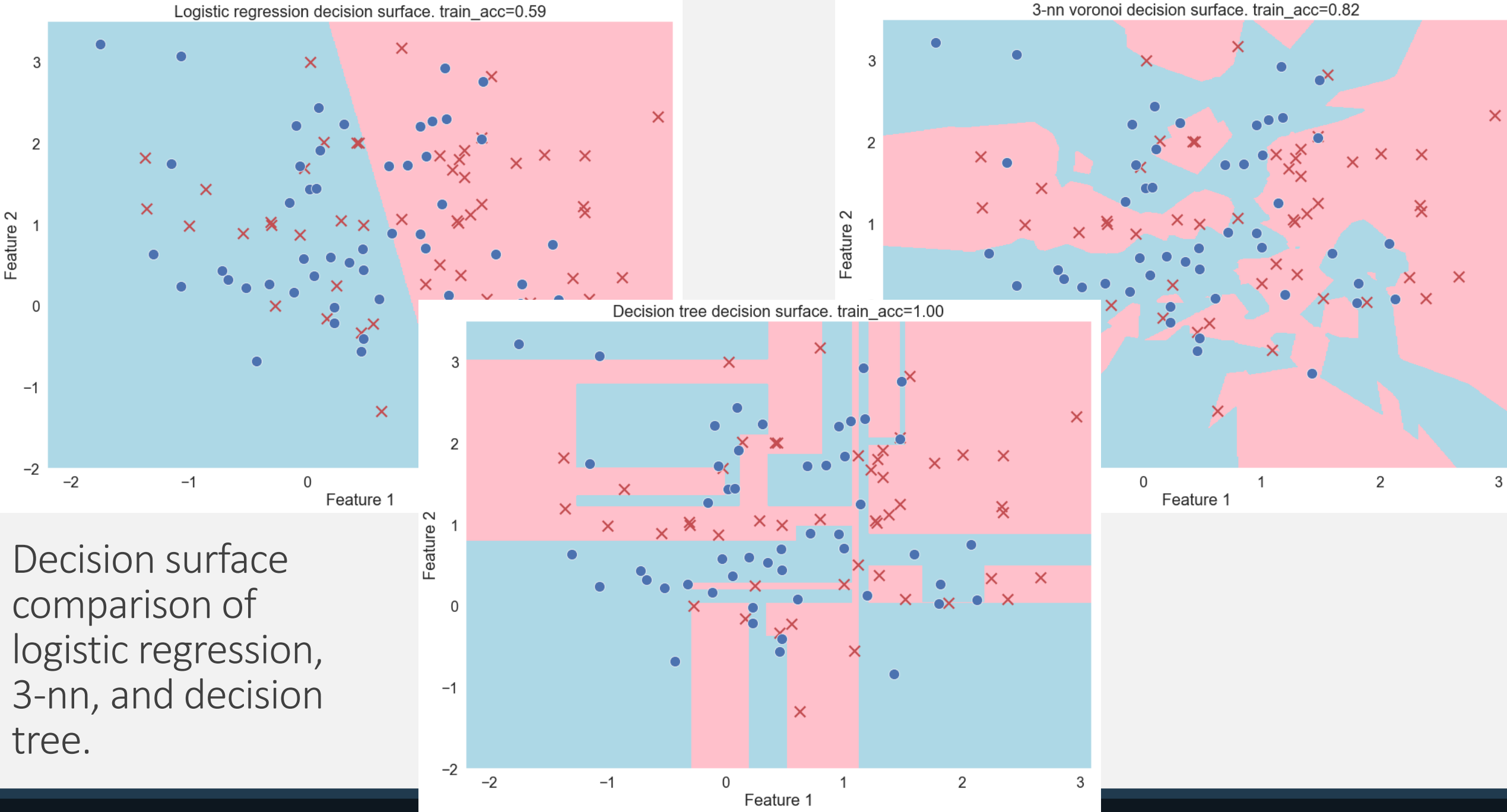
Logistic regression 2 feature example



Plane animation from [link](#)

Logistic regression can be **viewed** as optimizing in the sigmoid space $\sigma(x^T w)$ with fixed sigmoid slope and pushing the classification points to the limits by adjusting the weights.

Corresponding **view** is that every weight setting is a *sigmoidal* curve in the feature space (its slope and positioning).



Formalism [1]

- We have a **binary classification** task where an instance (x_i, y_i) is such that $x_i \in \mathbb{R}^n$ is a vector with n features, and $y_i \in \{0, 1\}$ is the binary label.
- Note that we will denote x_{ij} as the j th feature of the i th instance (example). So, $x_{3,9}$ refers to the 9th feature value of the 3rd instance in our dataset.
- We **assume** that the label y_i can be approximated by a linearly weighted combination of the input features. However, we use the sigmoid function:

$$\begin{aligned}\hat{y}_i &= \sigma(x_i^T w) \\ &= \frac{1}{1 + e^{-x_i^T w}} = \frac{1}{1 + e^{-(w_0 + w_1 x_{i1} + w_2 x_{i2} + \dots + w_n x_{in})}}\end{aligned}$$

Formalism [2]

- The sigmoid function will transform $x_i^T w$ to the range $(0, 1)$.
Large values will go to 1 and small (negative) values will go to 0.
$$\begin{aligned}\hat{y}_i &= \text{Prob}(y_i = 1 \mid x_i) \\ &= \sigma(x_i^T w) \\ &= \frac{1}{1 + e^{-x_i^T w}}\end{aligned}$$
- Without going into the details, note that logistic regression is predicting $\text{Prob}(y_i = 1 \mid x_i)$, that is the probability of class 1.
- How do we get final binary predictions? After finding the best weights w^* , we use a threshold value $\gamma \in [0, 1]$:
$$\hat{y}_i = \begin{cases} 0 & \text{if } \sigma(x_i^T w) < \gamma \\ 1 & \text{if } \sigma(x_i^T w) \geq \gamma \end{cases}$$
- Makes sense to use threshold of $\lambda = 0.5$. However, may want to choose a different threshold depending on the metric you want to optimize (covered later in course).

Summary of Logistic Regression Direct Optimization Formulation

- Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ for a binary classification problem (i.e. labels are $\{0, 1\}$). We consider predictions of the form:

$$\begin{aligned}\hat{y} &= \sigma(x^T w) \\ &= \frac{1}{1 + e^{-x^T w}}\end{aligned}$$

- To Train:** we find w^* to minimize the logloss:

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} - \left[\sum_{i=1}^m y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w)) \right]$$

- To Predict:** we can output probability $\hat{y}_{new} = \sigma(x_{new}^T w^*)$ or a label prediction by thresholding.

Logistic Regression Statistical Formulation [1]

- The sigmoid function has the range $(0, 1)$. This resembles a probability function.
- In logistic regression, we model $y_i|x_i$ as a Bernoulli distribution:

$$y_i|x_i \sim \text{Bernoulli}(p)$$

- This says, that the distribution of the label y_i depends on the feature vector x_i . The specific distribution we are assuming is Bernoulli. When we say that $y_i|x_i \sim \text{Bernoulli}(p)$, then explicitly this means:

$$\begin{aligned} \text{Prob}(y_i = 1|x_i) &= p \\ \text{Prob}(y_i = 0|x_i) &= 1 - p \end{aligned}$$

Equivalently

$$\text{Prob}(y_i|x_i) = p^{y_i} (1 - p)^{1-y_i}$$

Logistic Regression Statistical Formulation [2]

$$y_i|x_i \sim \text{Bernoulli}(p)$$

$$\text{Prob}(y_i|x_i) = p^{y_i}(1-p)^{1-y_i}$$

- So we're saying that the probability that $y_i = 1$ given x_i is the value of p . What value should we set p to?
- We can set $p = \sigma(x_i^T w)$. This way, if $x_i^T w$ is large, then y_i is close to 1, if $x_i^T w$ is small (negative), then y_i is close to 0.
- Thus, in logistic regression, the statistical formulation is:

$$y_i|x_i \sim \text{Bernoulli}(\sigma(x_i^T w))$$

$$\text{Prob}(y_i|x_i) = \sigma(x_i^T w)^{y_i}(1 - \sigma(x_i^T w))^{1-y_i}$$

- With this formulation, then our prediction gives us the probability of $y_i = 1|x_i$:

$$\begin{aligned}\hat{y}_i &= \sigma(x_i^T w) \\ &= \text{Prob}(y_i = 1|x_i)\end{aligned}$$

Where does logloss come from? [1]

$$y_i|x_i \sim \text{Bernoulli}(\sigma(x_i^T w))$$

$$\text{Prob}(y_i|x_i) = \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i}$$

$$\hat{y}_i = \sigma(x_i^T w) = \text{Prob}(y_i = 1|x_i)$$

- Use Maximum Likelihood Estimation (MLE) to find w that maximizes the probability of the training data.
- Given the training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ for a binary classification task, and assuming the Bernoulli setup, then the probability of the training data is given by:

$$\begin{aligned}\text{Prob}(\text{Train}) &= \text{Prob}[(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)] \\ &= \text{Prob}(x_1, y_1) \text{Prob}(x_2, y_2) \cdots \text{Prob}(x_m, y_m) \\ &= \prod_{i=1}^m \text{Prob}(x_i, y_i)\end{aligned}$$

Due to the independent and identically distributed (i.i.d) assumption, we can write the probability of the data as product of the instance probabilities.

Where does logloss come from? [2]

$$\begin{aligned} \text{Prob}(\text{Train}) &= \prod_{i=1}^m \text{Prob}(x_i, y_i) && \text{use rule } \text{Prob}(A, B) = \text{Prob}(B|A)\text{Prob}(A) \\ &= \prod_{i=1}^m \text{Prob}(y_i|x_i)\text{Prob}(x_i) && \text{use } \text{Prob}(y_i|x_i) = \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i} \\ &= \prod_{i=1}^m \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i} \text{Prob}(x_i) \end{aligned}$$

- In MLE, the goal is to find w (the parameter we can play around with), that maximizes this quantity (the probability of the data).

Where does logloss come from? [3]

- In MLE, the goal is to find w (the parameter we can play around with), that maximizes this quantity (the probability of the data).

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \text{Prob}(\text{Train})$$

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i} \text{Prob}(x_i)$$

We can drop $\text{Prob}(x_i)$ because it doesn't depend on w .

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i}$$

Some useful rules [1]

- Finding the maximum of a function $g(w)$ is the same as finding the maximum of $\log g(w)$.

$$\begin{aligned}w^* &= \arg \max_{w \in \mathbb{R}^{n+1}} g(w) \\&= \arg \max_{w \in \mathbb{R}^{n+1}} \log(g(w))\end{aligned}$$

- Finding the maximum of a function $g(w)$ is the same as finding the minimum of $-g(w)$.

$$\begin{aligned}w^* &= \arg \max_{w \in \mathbb{R}^{n+1}} g(w) \\&= \arg \min_{w \in \mathbb{R}^{n+1}} -g(w)\end{aligned}$$

Some useful rules [2]

- The log of products can be changed to sum of logs. If a and b are scalars, then:

$$\log(ab) = \log(a) + \log(b)$$

- The log of quotients can be changed to difference of logs. If a and b are scalars, then:

$$\log\left(\frac{a}{b}\right) = \log(a) - \log(b)$$

- Power rule of log. If a and b are scalars, then:

$$\log(a^b) = b \log(a)$$

Where does logloss come from? [4]

- In MLE, the goal is to find w (the parameter we can play around with), that maximizes this quantity (the probability of the data).

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i}$$

Use rule from [previous slides](#)

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \log \left(\prod_{i=1}^m \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i} \right)$$

Use log product rule

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m \log \left(\sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i} \right)$$

Use log product rule again

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m \log \left(\sigma(x_i^T w)^{y_i} \right) + \log \left((1 - \sigma(x_i^T w))^{1-y_i} \right)$$

Where does logloss come from? [5]

- In MLE, the goal is to find w (the parameter we can play around with), that maximizes this quantity (the probability of the data).

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m \log \left(\sigma(x_i^T w)^{y_i} \right) + \log \left((1 - \sigma(x_i^T w))^{1-y_i} \right)$$

Use log power rule

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m y_i \log \left(\sigma(x_i^T w) \right) + (1 - y_i) \log \left(1 - \sigma(x_i^T w) \right)$$

Change maximization to minimization by taking negative

$$= \arg \min_{w \in \mathbb{R}^{n+1}} - \sum_{i=1}^m y_i \log \left(\sigma(x_i^T w) \right) + (1 - y_i) \log \left(1 - \sigma(x_i^T w) \right)$$

Where does logloss come from? [2]

$$y_i|x_i \sim \text{Bernoulli}(\sigma(x_i^T w))$$

$$\text{Prob}(y_i|x_i) = \sigma(x_i^T w)^{y_i} (1 - \sigma(x_i^T w))^{1-y_i}$$

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \text{Prob}(\text{Train})$$

$$= \arg \min_{w \in \mathbb{R}^{n+1}} - \left[\sum_{i=1}^m y_i \log (\sigma(x_i^T w)) + (1 - y_i) \log (1 - \sigma(x_i^T w)) \right]$$

- Using the MLE formulation, and using rules, you end up with this loss minimization problem.
- This specific loss is called the **logloss** or **cross-entropy loss**.
- Finding the minimum of this loss function is the goal in logistic regression.

Summary of Logistic Regression Formulation [1]

- Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ for a binary classification problem (i.e. labels are $\{0, 1\}$). We consider predictions of the form:

$$\begin{aligned}\hat{y} &= \sigma(x^T w) \\ &= \frac{1}{1 + e^{-x^T w}}\end{aligned}$$

- Goal:** Find minimum of the logloss:

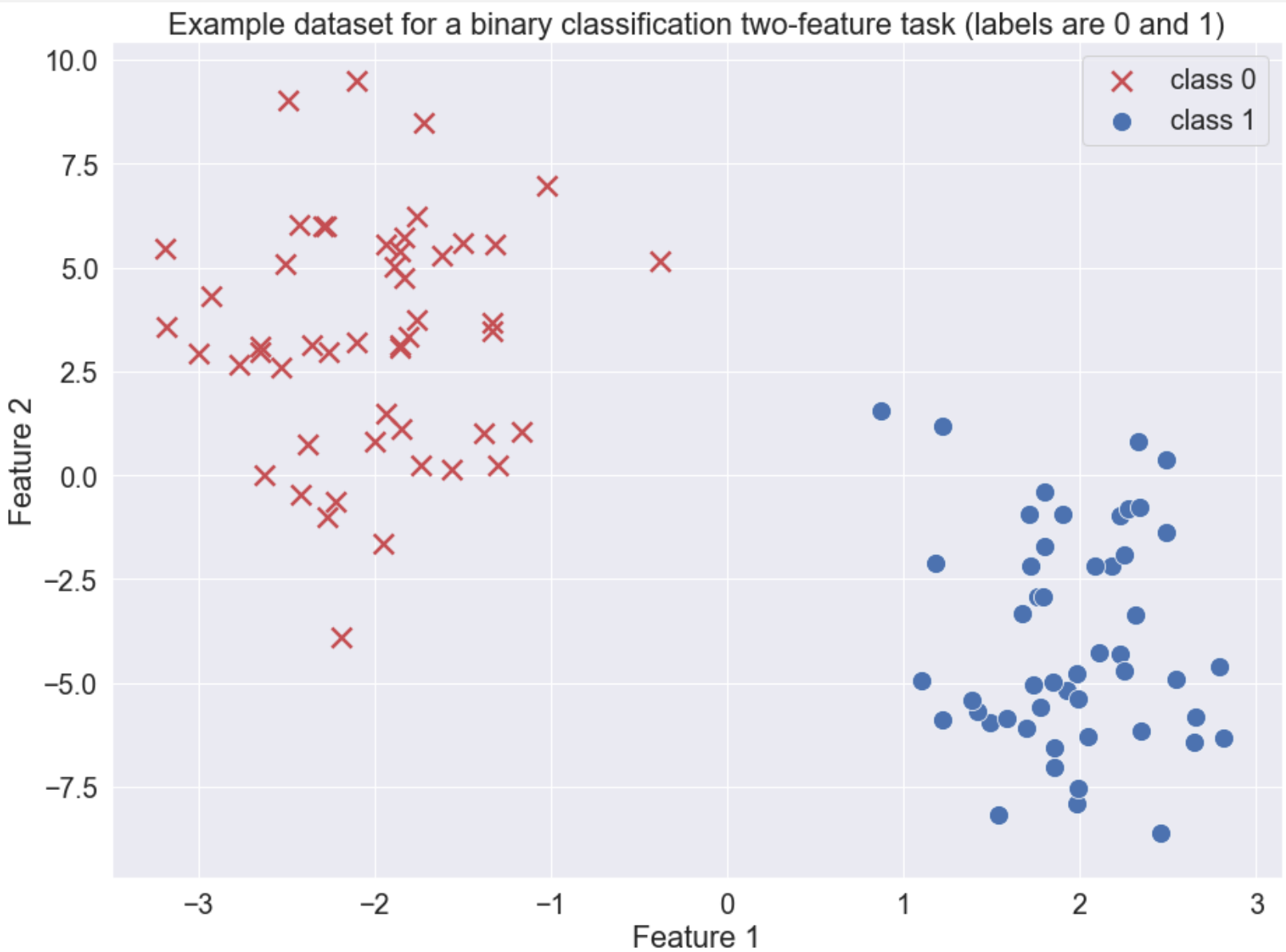
$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} - \left[\sum_{i=1}^m y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w)) \right]$$

Summary of Logistic Regression Formulation [2]

$$\begin{aligned}\hat{y} &= \sigma(x^T w) \\ &= \frac{1}{1 + e^{-x^T w}}\end{aligned}$$

- Note that the sigmoid function $\sigma()$ performs a non-linear transformation of $x^T w$. However, logistic regression is still considered linear because the interaction between the weights and features is still linear in the weights.

Logistic Regression – Example Two-feature Problem [1]



- Example of a binary classification dataset with two features.
- We want to apply logistic regression:

$$\hat{y} = \sigma(w_0 + w_1 feat1 + w_2 feat2)$$

- We find the minimum of the logloss:

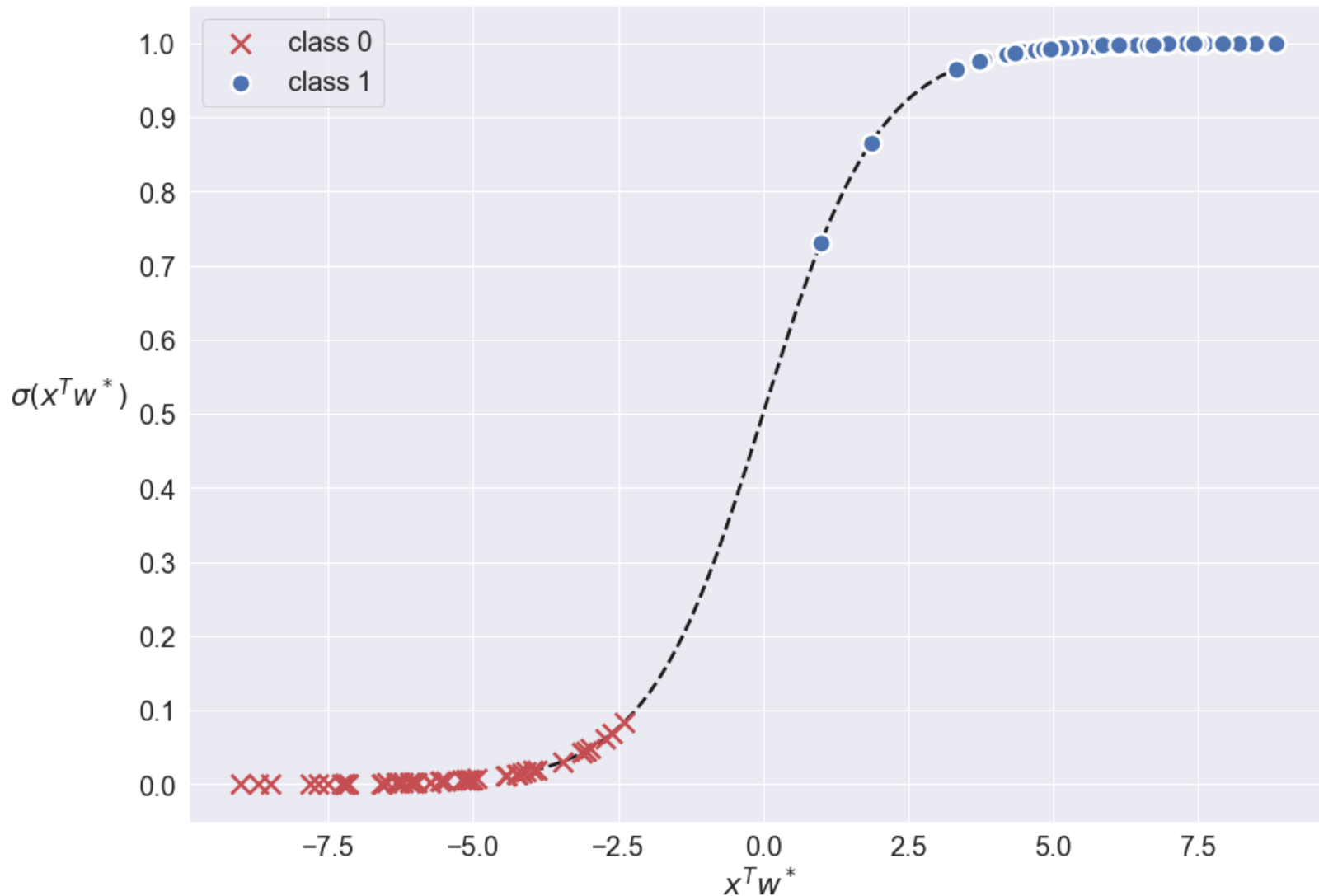
$$w^* = [0.04, 1.94, -0.47]$$

- What does this solution look like?

Logistic Regression – Example Two-feature Problem [2]

Sigmoid transformation of the two-feature binary dataset using the Logistic Regression solution

$$w^* = [0.04, 1.94, -0.47]$$



$$w^* = [0.04, 1.94, -0.47]$$

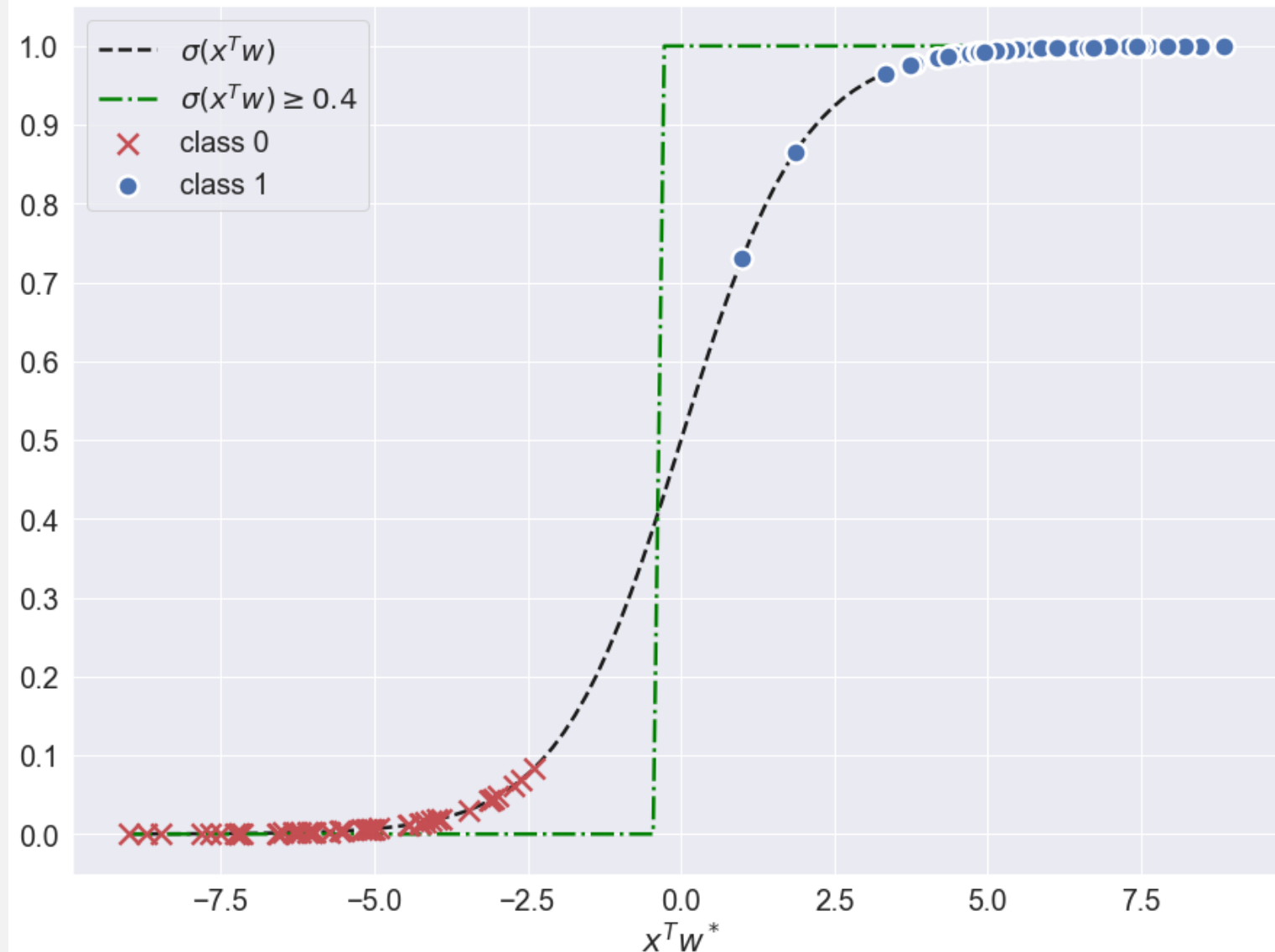
$$\hat{y} = \sigma(x^T w^*)$$

- This is what logistic regression does, it finds weights to *transform* the input features to the sigmoid curve. But in such a way that creates a separation between the two classes.
- We can see from the solution weights, we get a nice separation between class 0 and class 1.

Logistic Regression – Example Two-feature Problem [3]

Sigmoid transformation of the two-feature binary dataset using the Logistic Regression solution

$$w^* = [0.04, 1.94, -0.47]$$



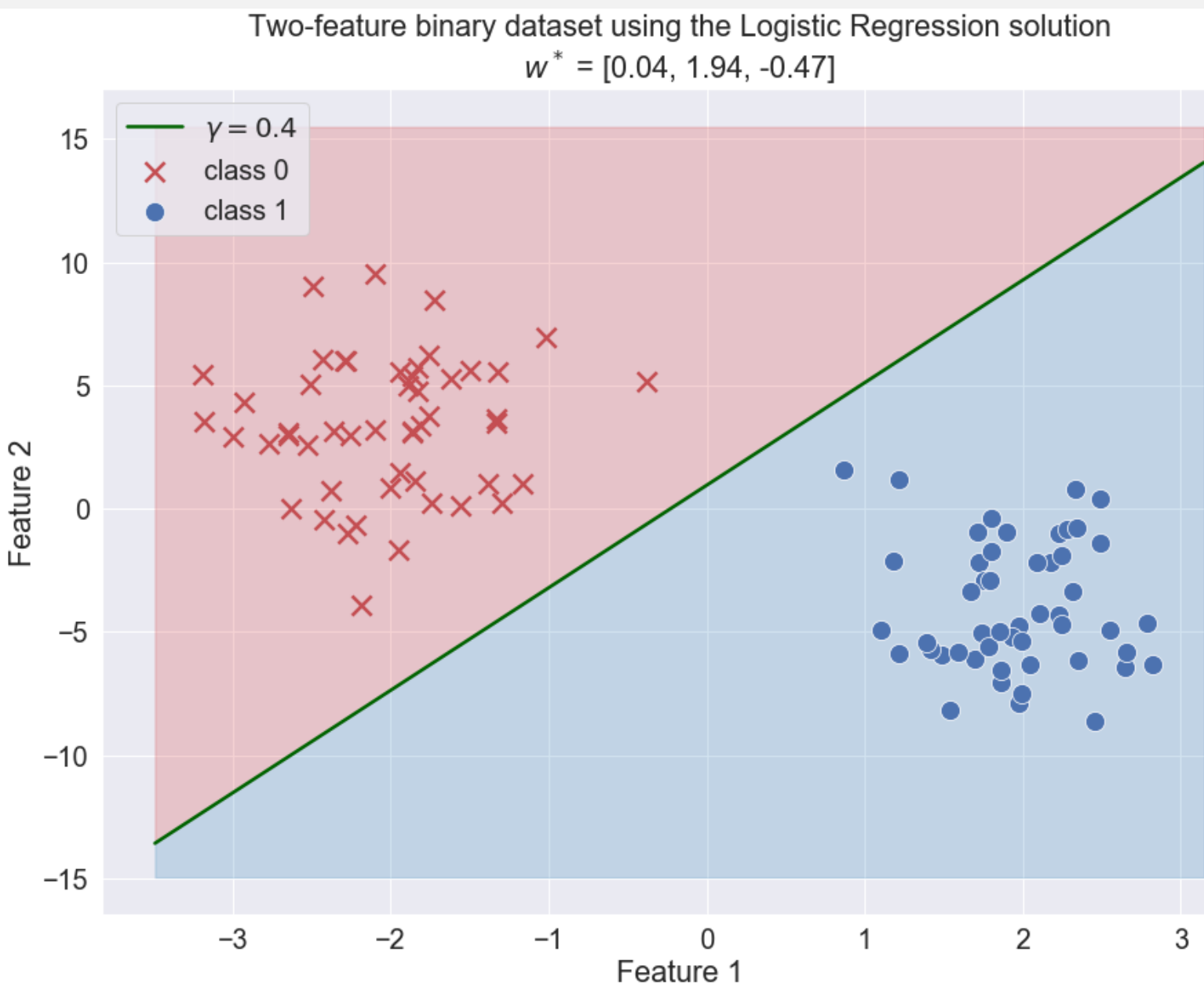
$$w^* = [0.04, 1.94, -0.47]$$

$$\hat{y} = \sigma(x^T w^*)$$

- Recall that the sigmoid function is predicting the probability of class 1.
- We can force a class prediction via thresholding:

$$\hat{y} = \begin{cases} 0 & \text{if } \sigma(x^T w^*) < \gamma \\ 1 & \text{if } \sigma(x^T w^*) \geq \gamma \end{cases}$$

- Let's say we use $\gamma = 0.4$



$$w^* = [0.04, 1.94, -0.47]$$

$$\hat{y} = \sigma(x^T w^*)$$

$$\hat{y} = \begin{cases} 0 & \text{if } \sigma(x^T w^*) < 0.4 \\ 1 & \text{if } \sigma(x^T w^*) \geq 0.4 \end{cases}$$

- After thresholding, this can be mapped back to a decision boundary in the original feature space.

$$\sigma(x^T w^*) = 0.4 \text{ when } x^T w^* = \sigma^{-1}(0.4)$$

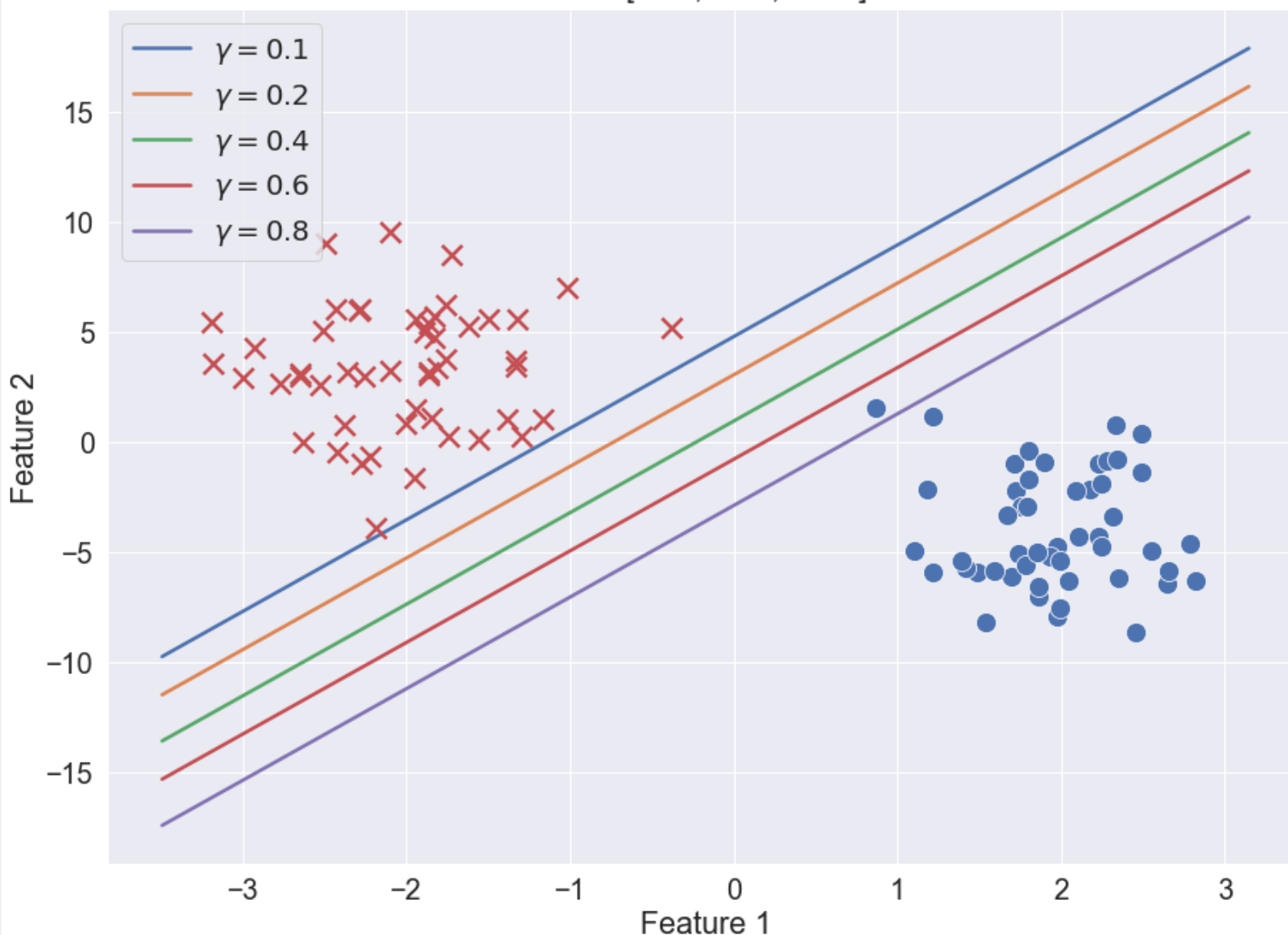
$$w_0 + w_1 \text{feat1} + w_2 \text{feat2} = \sigma^{-1}(0.4)$$

$$\text{feat2} = \frac{\sigma^{-1}(0.4) - w_0}{w_2} - \frac{w_1}{w_2} \text{feat1}$$

- This defines a line with a slope and intercept. We can plot this map as shown by the green line. This also defines the decision surface (i.e. red and blue regions).

Two-feature binary dataset using the Logistic Regression solution

$$w^* = [0.04, 1.94, -0.47]$$



$$w^* = [0.04, 1.94, -0.47]$$

$$\hat{y} = \begin{cases} 0 & \text{if } \sigma(x^T w^*) < \gamma \\ 1 & \text{if } \sigma(x^T w^*) \geq \gamma \end{cases}$$

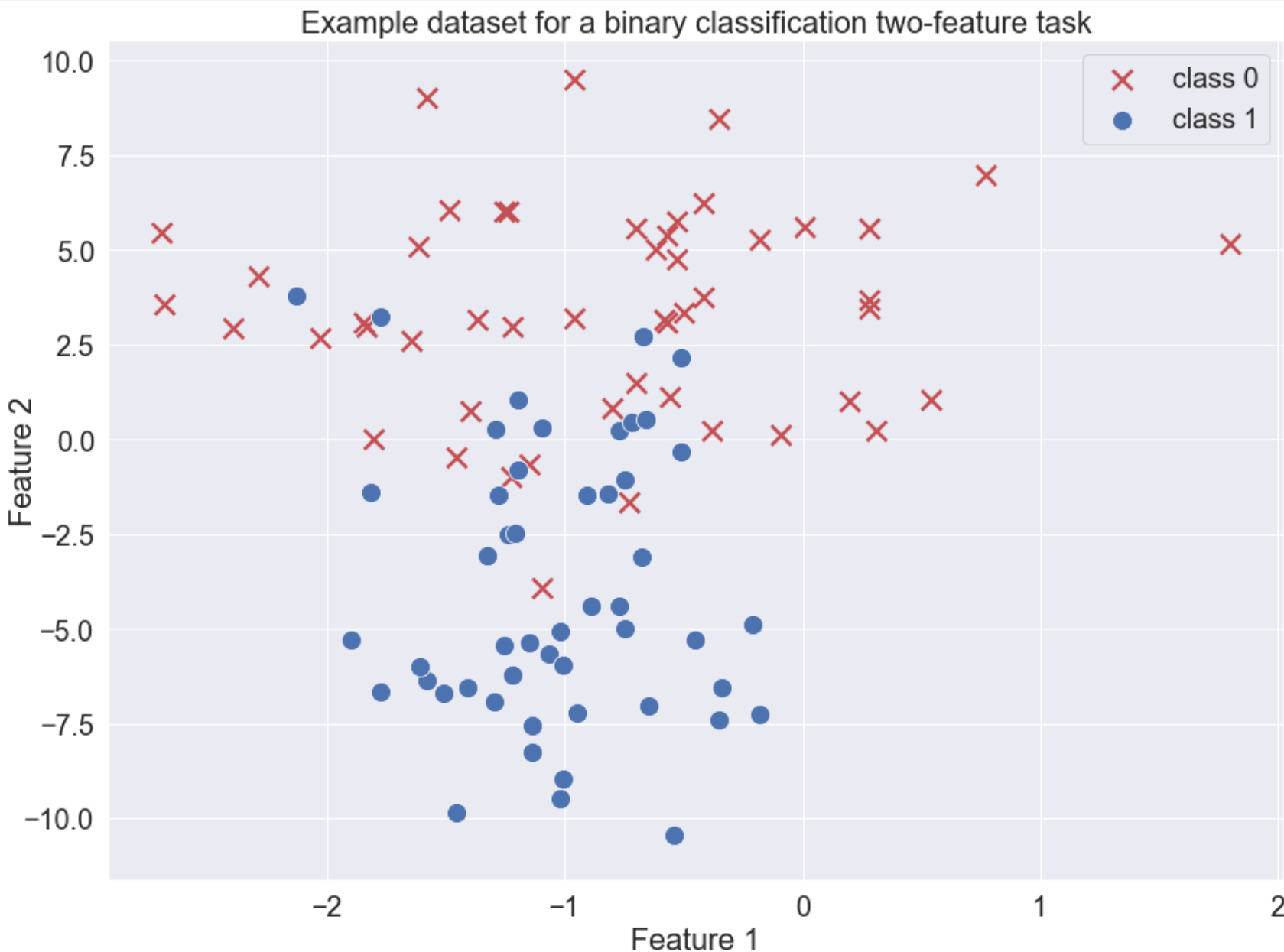
$$\sigma^{-1}(z) = \ln(z) - \ln(1 - z)$$

$$w_0 + w_1 \text{feat1} + w_2 \text{feat2} = \sigma^{-1}(\gamma)$$

$$\text{feat2} = \frac{\sigma^{-1}(\gamma) - w_0}{w_2} - \frac{w_1}{w_2} \text{feat1}$$

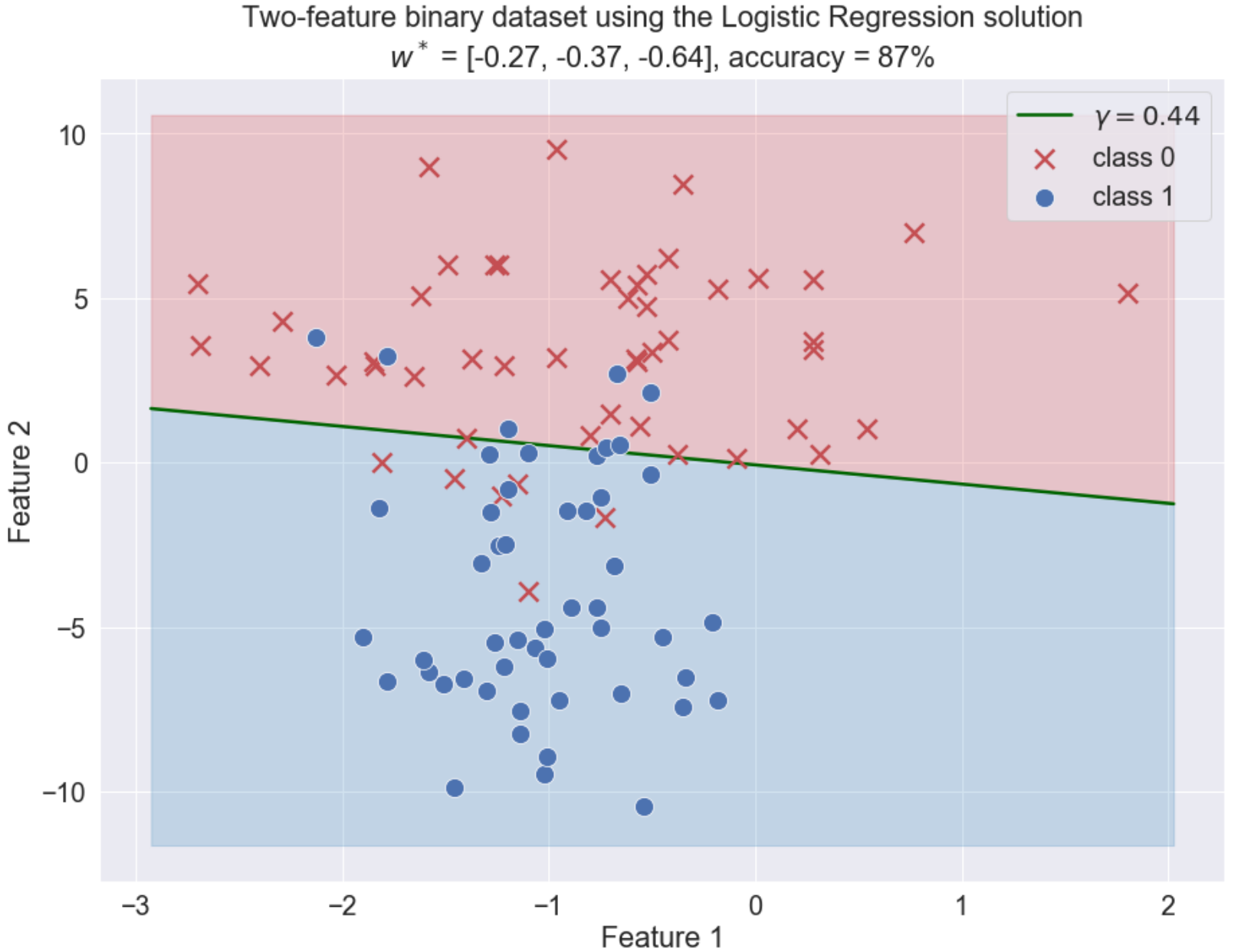
- For a given w^* , different thresholds move the decision boundary.

Logistic Regression – Example Two-feature Non-Linearly Separable [1]



- The previous example was **linearly separable**.
- However, in this example we cannot draw a line to perfectly separate the two classes.
- This problem is **non-linearly** separable.
- Nonetheless, we can still apply logistic regression to find the weights that minimize the logloss.

Logistic Regression – Example Two-feature Non-Linearly Separable [3]



Minimizing the Logloss

Summary of Logistic Regression Formulation

- Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ for a binary classification problem (i.e. labels are $\{0, 1\}$). We consider predictions of the form:

$$\begin{aligned}\hat{y} &= \sigma(x^T w) \\ &= \frac{1}{1 + e^{-x^T w}}\end{aligned}$$

- Goal:** Find minimum of the logloss:

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} - \left[\sum_{i=1}^m y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w)) \right]$$

Minimizing the LogLoss [1]

- Take the partial derivative of the loss with respect to w :

$$\frac{\partial \ell(w)}{\partial w} = \frac{\partial}{\partial w} - \left[\sum_{i=1}^m y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w)) \right] = \begin{bmatrix} \frac{\partial \ell(w)}{\partial w_0} \\ \frac{\partial \ell(w)}{\partial w_1} \\ \vdots \\ \frac{\partial \ell(w)}{\partial w_n} \end{bmatrix}$$

$$\frac{\partial \ell(w)}{\partial w_j} = \frac{\partial - \left[\sum_{i=1}^m y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w)) \right]}{\partial w_j}$$

Do the derivation on your own

$$\text{Recall } \frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$$

Minimizing the LogLoss [2]

Recall $\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$

$$\begin{aligned}\frac{\partial \ell(w)}{\partial w_j} &= \frac{\partial - \sum_{i=1}^m [y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w))]}{\partial w_j} \\&= - \frac{\partial \sum_{i=1}^m y_i \log(\sigma(x_i^T w))}{\partial w_j} - \frac{\partial \sum_{i=1}^m (1 - y_i) \log(1 - \sigma(x_i^T w))}{\partial w_j} \\&= - \sum_{i=1}^m y_i \frac{1}{\sigma(x_i^T w)} \sigma(x_i^T w)(1 - \sigma(x_i^T w)) x_{ij} - \sum_{i=1}^m (1 - y_i) \frac{1}{1 - \sigma(x_i^T w)} (0 - (1 - \sigma(x_i^T w)) \sigma(x_i^T w) x_{ij}) \\&= - \sum_{i=1}^m y_i x_{ij} - y_i x_{ij} \sigma(x_i^T w) - \sum_{i=1}^m y_i x_{ij} \sigma(x_i^T w) - x_{ij} \sigma(x_i^T w)\end{aligned}$$

Minimizing the LogLoss [3]

$$\begin{aligned}\frac{\partial \ell(w)}{\partial w_j} &= - \sum_{i=1}^m y_i x_{ij} - y_i x_{ij} \sigma(x_i^T w) - \sum_{i=1}^m y_i x_{ij} \sigma(x_i^T w) - x_{ij} \sigma(x_i^T w) \\&= - \sum_{i=1}^m y_i x_{ij} - y_i x_{ij} \sigma(x_i^T w) + y_i x_{ij} \sigma(x_i^T w) - x_{ij} \sigma(x_i^T w) \\&= - \sum_{i=1}^m y_i x_{ij} - x_{ij} \sigma(x_i^T w) = - \sum_{i=1}^m (y_i - \sigma(x_i^T w)) x_{ij} = \sum_{i=1}^m (\sigma(x_i^T w) - y_i) x_{ij} \\&\implies \frac{\partial \ell(w)}{\partial w_j} = \sum_{i=1}^m (\sigma(x_i^T w) - y_i) x_{ij}\end{aligned}$$

Minimizing the LogLoss [4]

$$\frac{\partial \ell(w)}{\partial w_j} = \sum_{i=1}^m (\sigma(x_i^T w) - y_i) x_{ij} \implies \frac{\partial \ell(w)}{\partial w} = \sum_{i=1}^m (\sigma(x_i^T w) - y_i) x_i$$

$$\implies \frac{\partial \ell(w)}{\partial w} = X^T (\sigma(Xw) - Y)$$

- This is similar to the gradient of the squared loss, except here we apply an elementwise sigmoid operation to Xw .

Minimizing the LogLoss [5]

$$\frac{\partial \ell(w)}{\partial w} = X^T (\sigma(Xw) - Y)$$

- A logical next step would be to set this gradient to **0** and solve for w . However, there is no closed form solution due to the sigmoid.
- But since we have the gradient in a clean form, we can apply gradient descent to get the solution.

Minimizing the LogLoss with Gradient Descent

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} - \left[\sum_{i=1}^m y_i \log(\sigma(x_i^T w)) + (1 - y_i) \log(1 - \sigma(x_i^T w)) \right]$$

$$\frac{\partial \ell(w)}{\partial w} = X^T (\sigma(Xw) - Y)$$

- Randomly initialize $\hat{w}$. Set $\alpha \in (0, 1)$. For N iterations or until the loss stops decreasing, perform the following update:

$$\begin{aligned} \hat{w} &:= \hat{w} - \alpha \frac{\partial \ell(\hat{w})}{\partial w} \\ &:= \hat{w} - \alpha X^T (\sigma(X\hat{w}) - Y) \end{aligned}$$

Remarks on Logistic Regression Solution [1]

- Thus, we can solve logistic regression using gradient descent. We can also apply mini-batch gradient descent.
- The logloss function is convex, so there is a global minimum.
- The stability issues of gradient descent from Linear Regression can also appear in Logistic Regression. Thus, you should rescale the features using Standard-Scaler before solving Logistic Regression.
- We can also apply regularization to the logloss as we did in Linear Regression.

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \logloss(w) + \lambda R(w)$$

Remarks on Logistic Regression Solution [2]

- We can also apply regularization to the logloss as we did in Linear Regression.

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \text{logloss}(w) + \lambda R(w)$$

- scikit-learn's LogisticRegression applies regularization by default.

Parameters:

penalty : {'l1', 'l2', 'elasticnet', 'none'}, default='l2'

Specify the norm of the penalty:

- 'none': no penalty is added;
- 'l2': add a L2 penalty term and it is the default choice;
- 'l1': add a L1 penalty term;
- 'elasticnet': both L1 and L2 penalty terms are added.

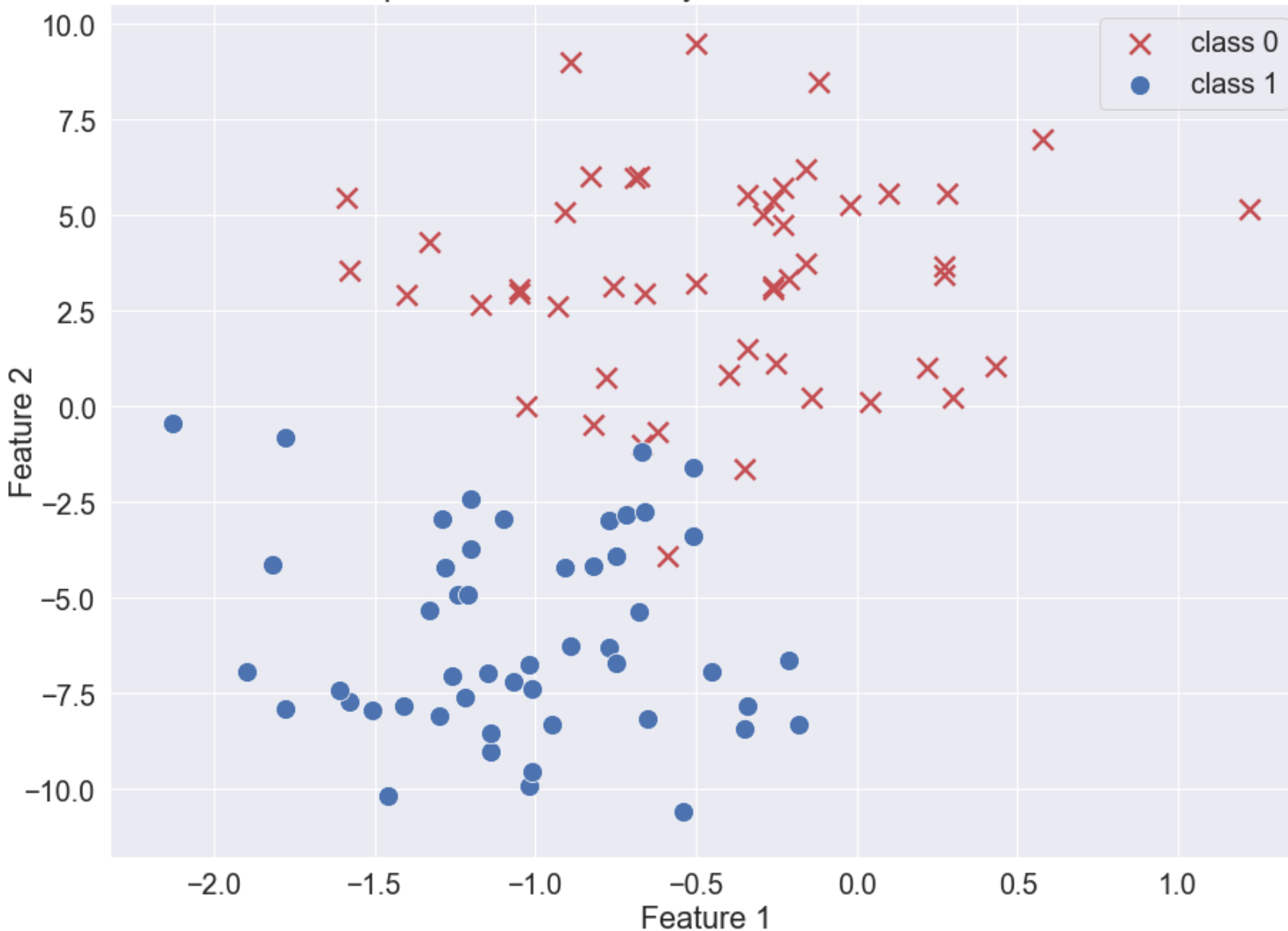
SGDClassifier in scikit-learn

```
class sklearn.linear_model.SGDClassifier(loss='hinge', *, penalty='l2',
alpha=0.0001, l1_ratio=0.15, fit_intercept=True, max_iter=1000,
tol=0.001, shuffle=True, verbose=0, epsilon=0.1, n_jobs=None,
random_state=None, learning_rate='optimal', eta0=0.0, power_t=0.5,
early_stopping=False, validation_fraction=0.1, n_iter_no_change=5,
class_weight=None, warm_start=False, average=False) # \[source\]
```

- [LogisticRegression](#) uses iterative solvers that are not exactly the same as gradient descent.
- [SGDClassifier](#) does SGD (batch size of 1). Also, has [partial_fit](#)(X_batch, y_batch) for you to use your own batch updates.

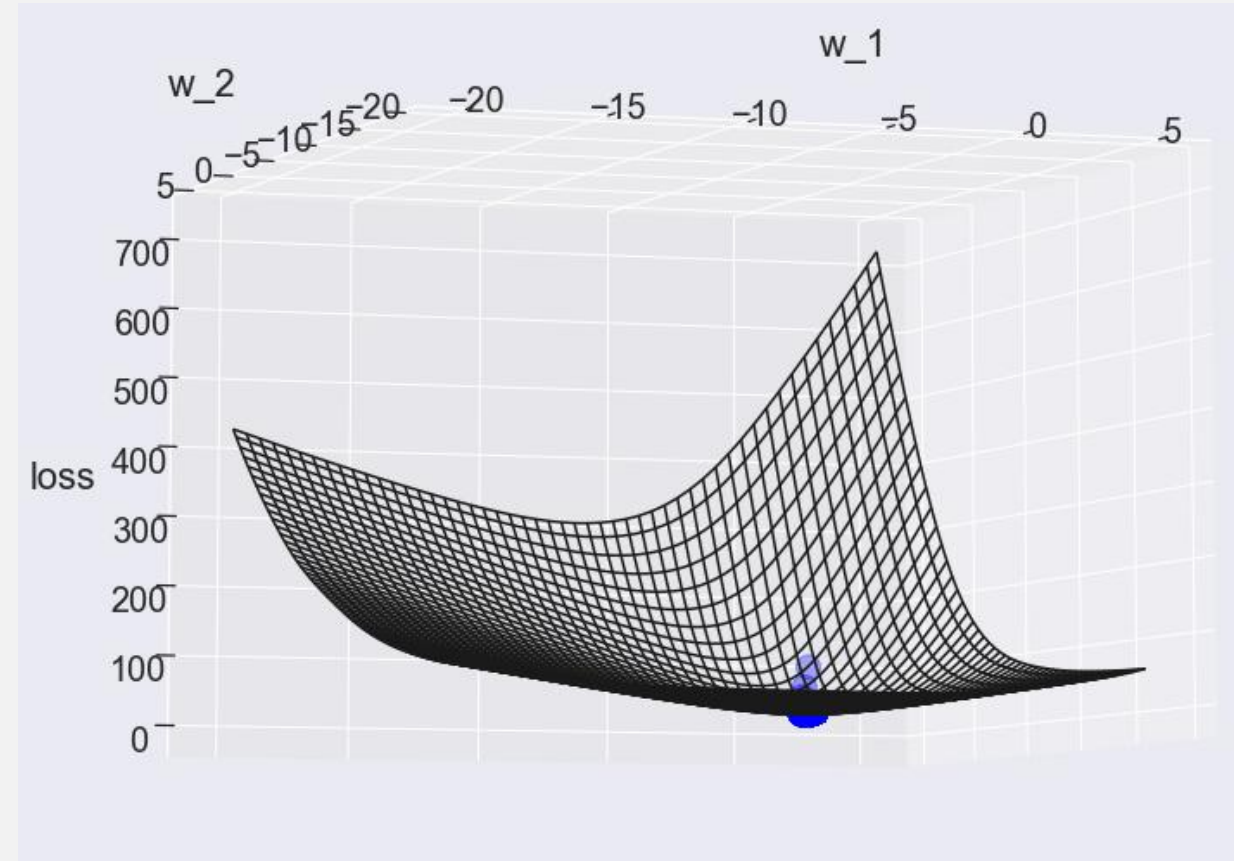
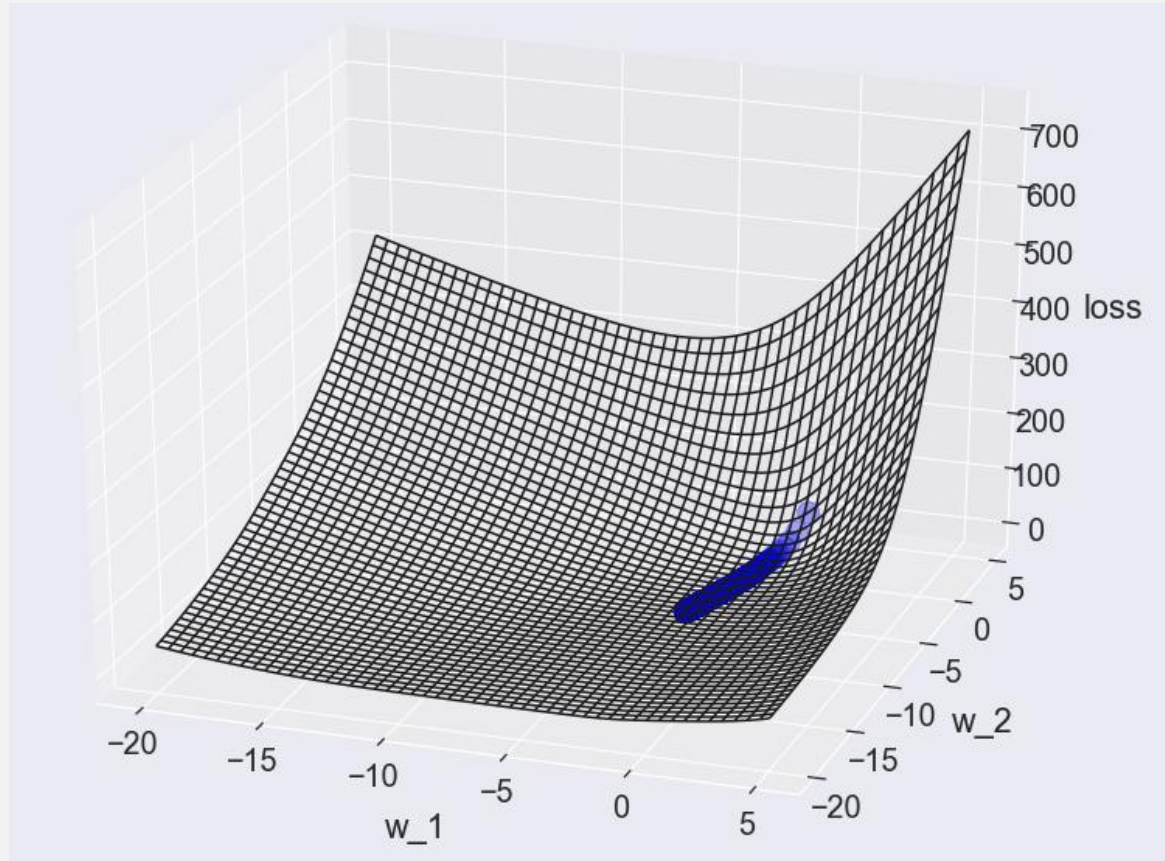
Logistic Regression – Example Two-feature Non-Linearly Separable

Example dataset for a binary classification two-feature task



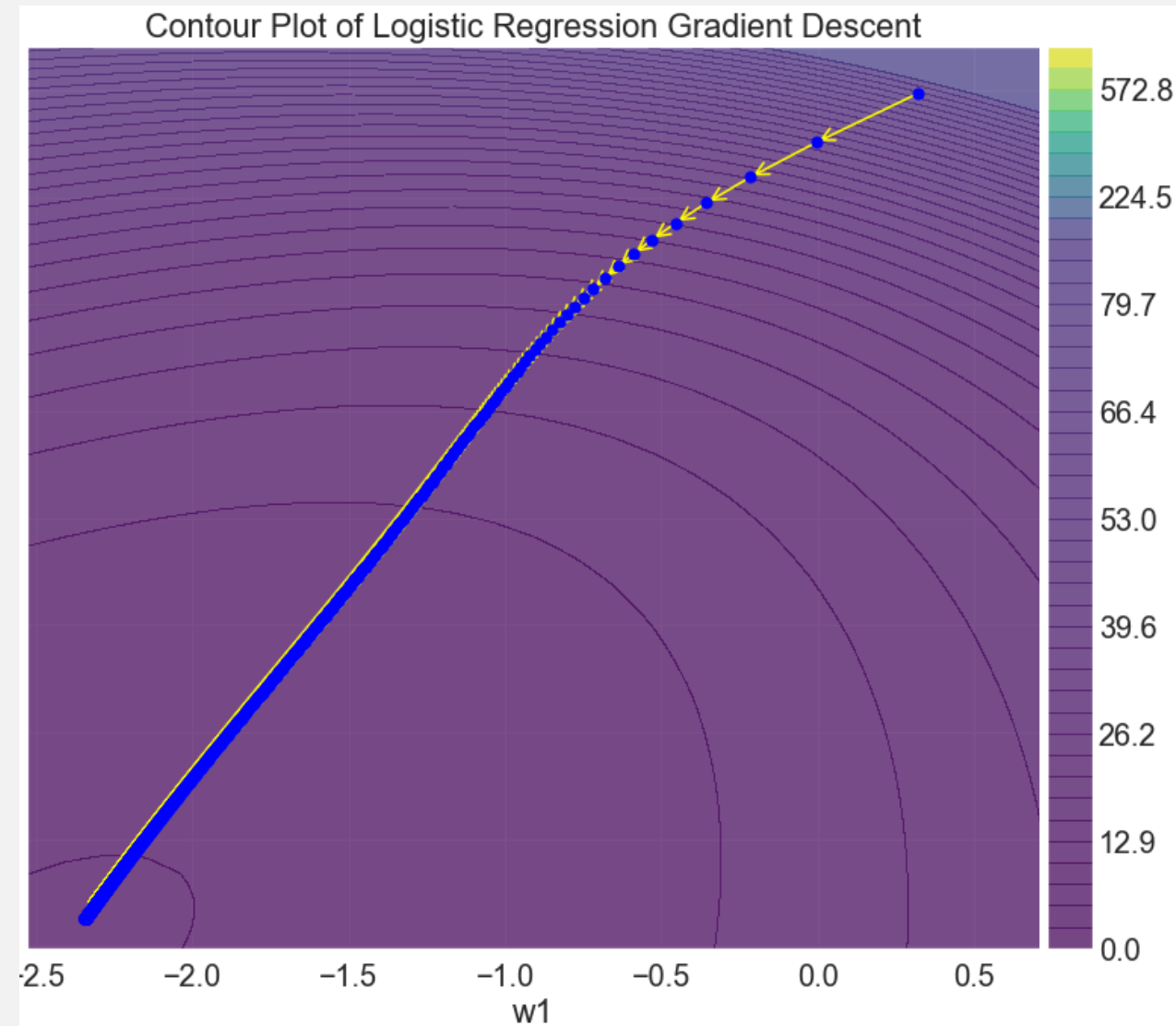
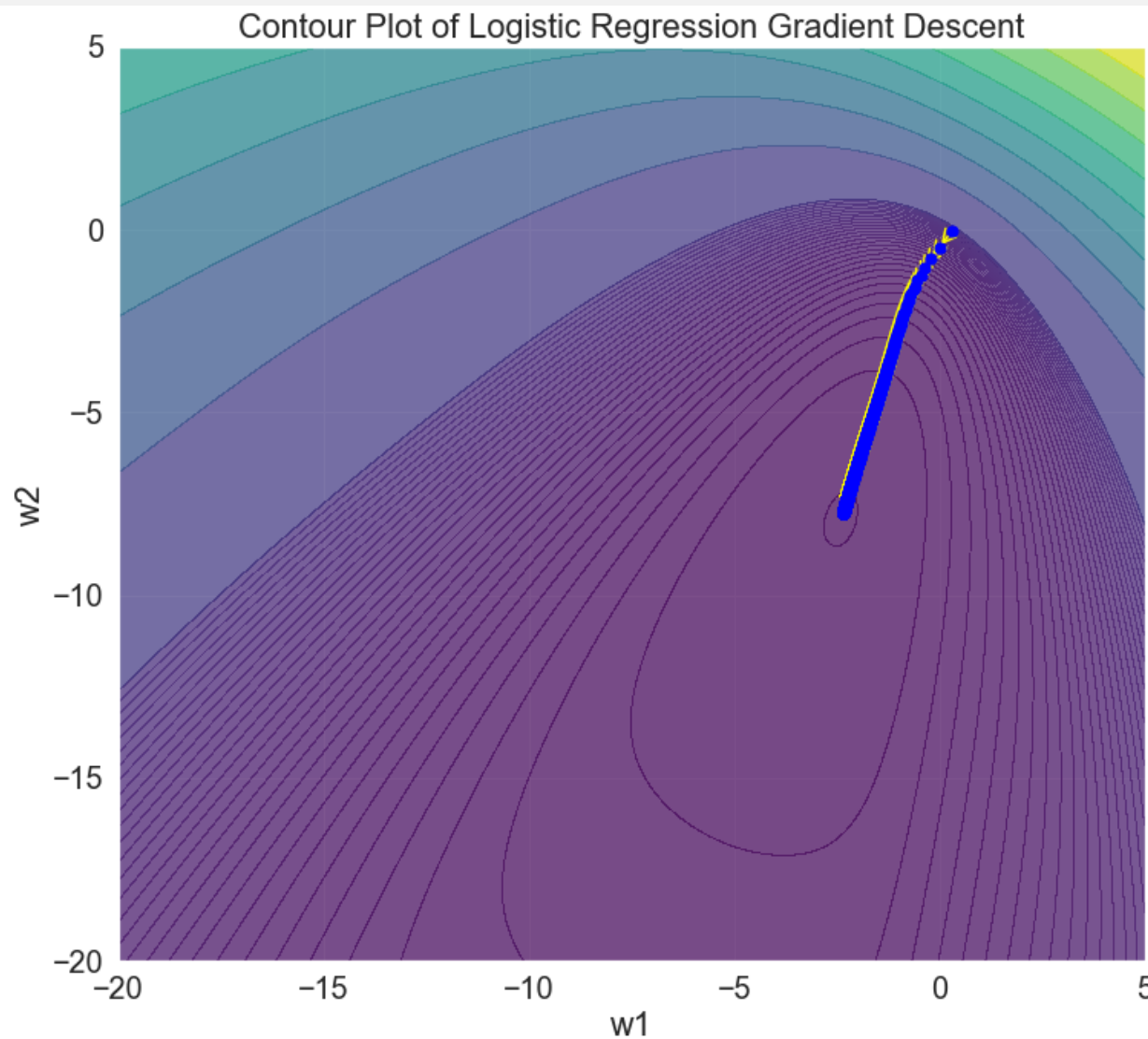
- Consider this non-linearly separable binary classification dataset.
- We'll apply gradient descent on the logloss.

Logistic Regression – LogLoss Visualization on a two-feature problem [1]

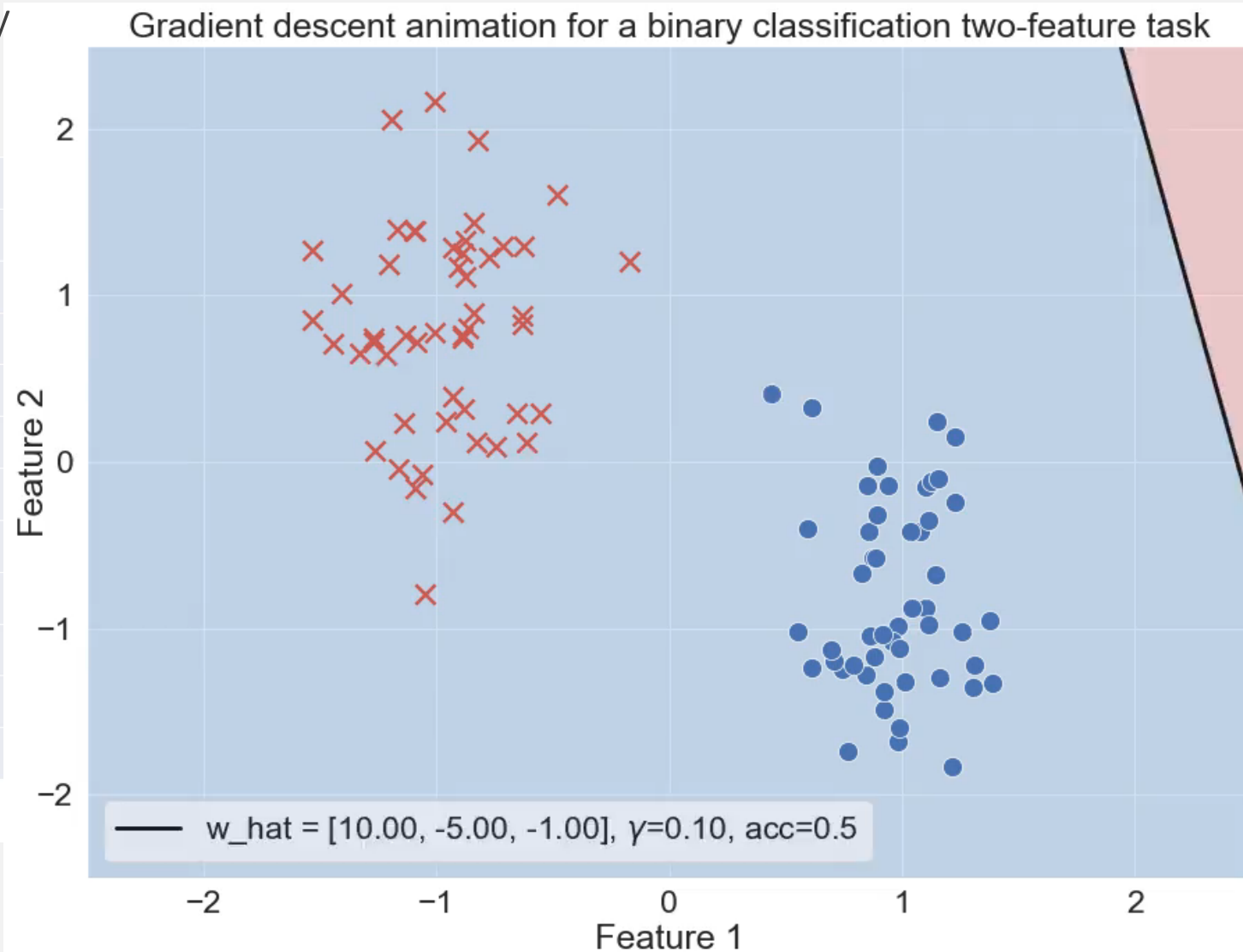
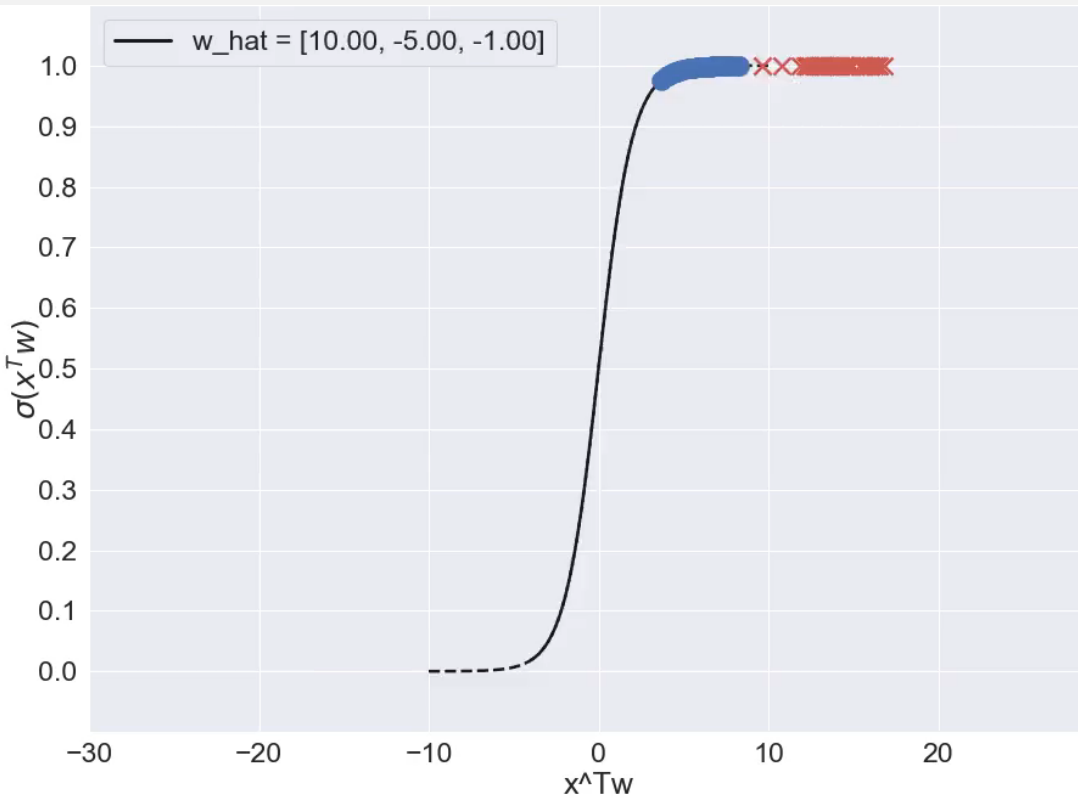


- We can visualize the logloss surface with respect to the two feature weights.
- The blue dots showcase the progression of gradient descent until it reaches the minimum.

Logistic Regression – LogLoss Visualization on a two-feature problem [2]



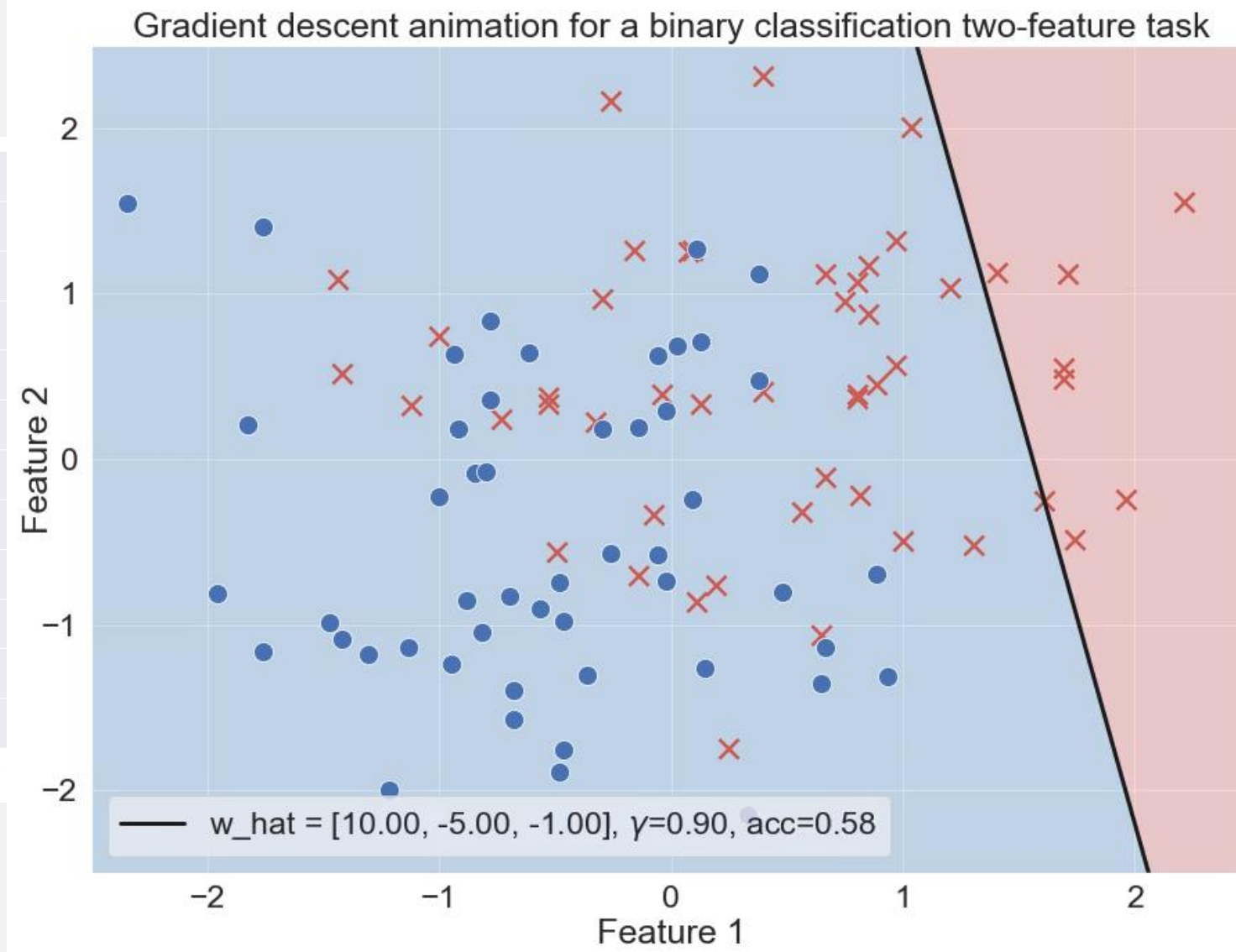
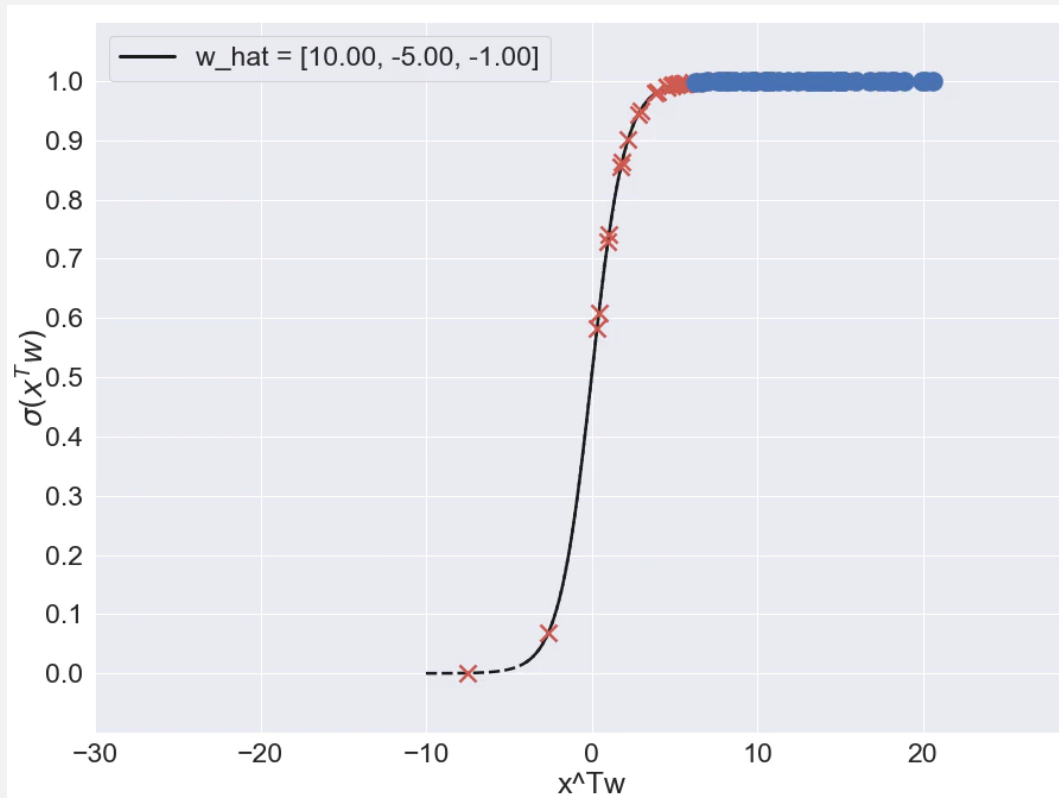
Logistic Regression GD Animation on linearly separable dataset



Logistic regression can be **viewed** as optimizing in the sigmoid space $\sigma(x^T w)$ with fixed sigmoid slope and pushing the classification points to the limits by adjusting the weights.

Corresponding **view** is that every weight setting is a *sigmoidal* curve in the feature space (its slope and positioning).

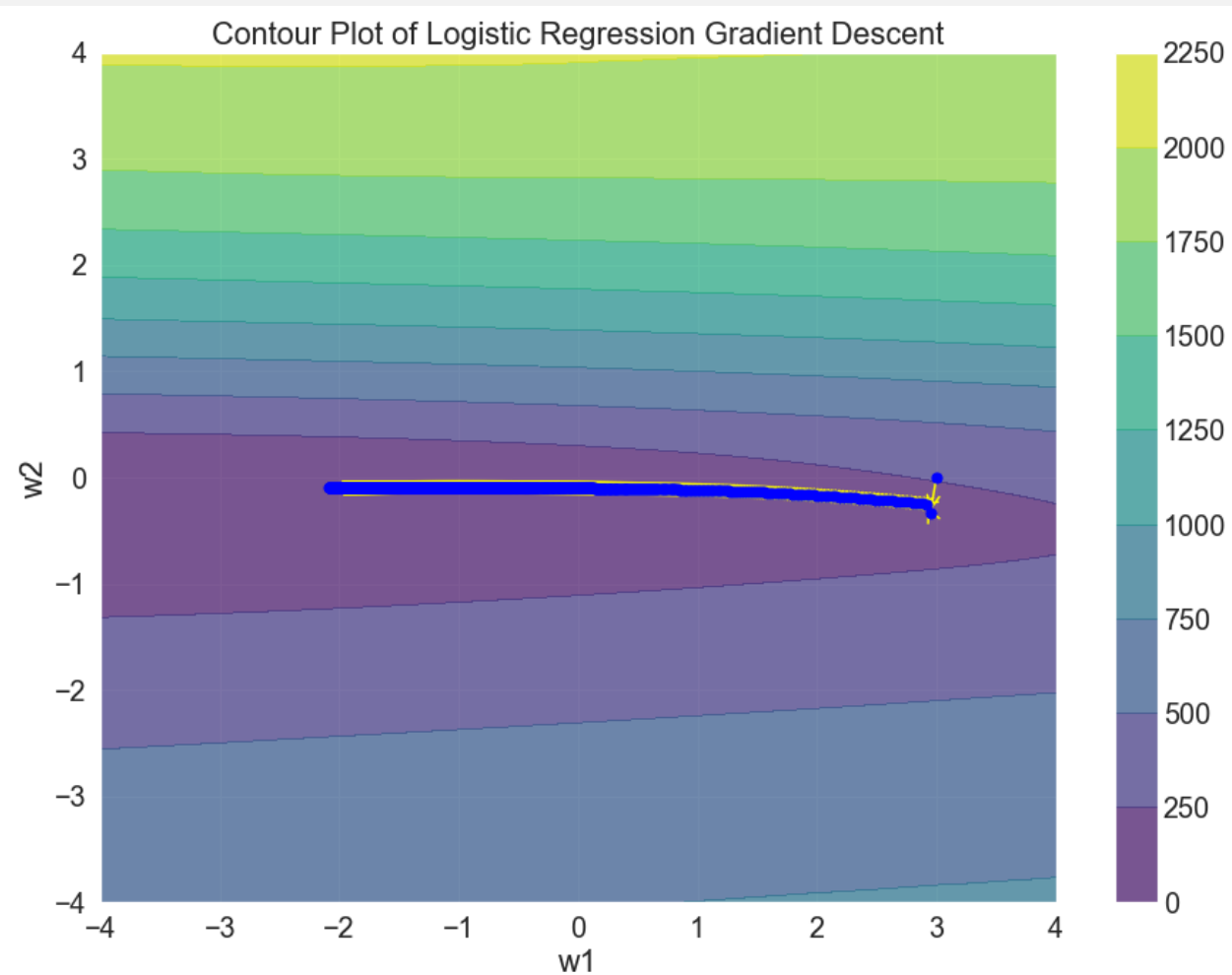
Logistic Regression GD Animation on NOT linearly separable dataset



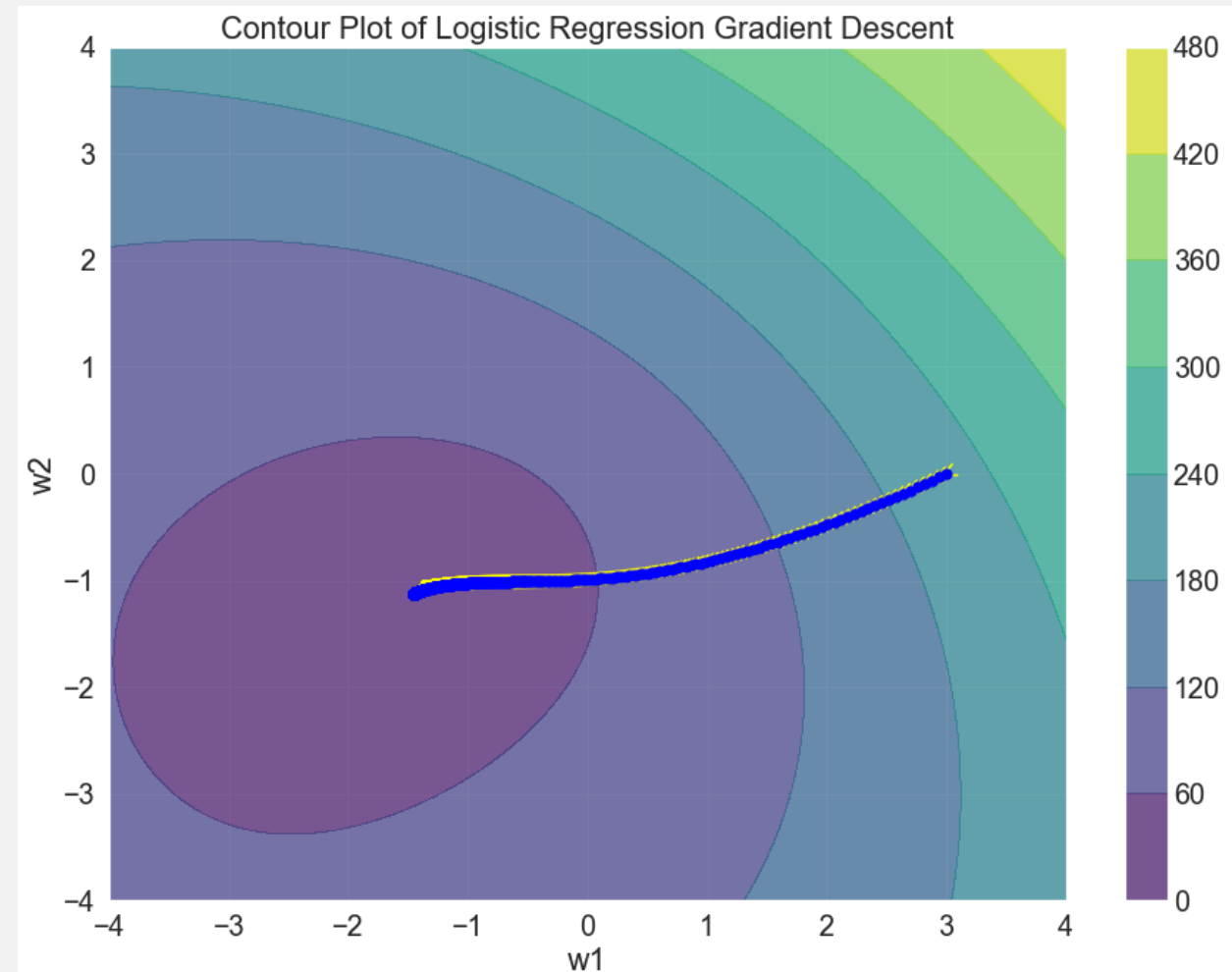
Logistic regression can be **viewed** as optimizing in the sigmoid space $\sigma(x^T w)$ with fixed sigmoid slope and pushing the classification points to the limits by adjusting the weights.

Corresponding **view** is that every weight setting is a *sigmoidal* curve in the feature space (its slope and positioning).

Feature Scaling in Logistic Regression



Without Feature Scaling



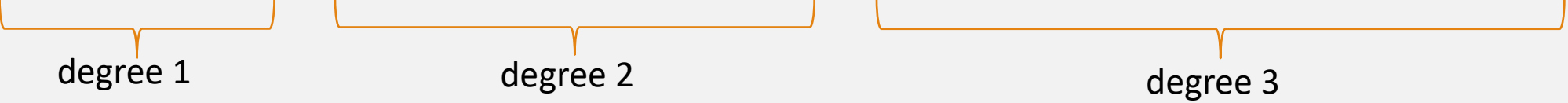
With Feature Scaling

Polynomial (Logistic) Regression

2-feature Polynomial Logistic Regression

- For an 2-feature problem and polynomial logistic regression of degree 3.

$$h(x_i) = \sigma(w_0 + w_1x_{i1} + w_2x_{i2} + w_3x_{i1}x_{i2} + w_4x_{i1}^2 + w_5x_{i2}^2 + w_6x_{i1}^2x_{i2} + w_7x_{i1}x_{i2}^2 + w_8x_{i1}^3 + w_9x_{i2}^3)$$

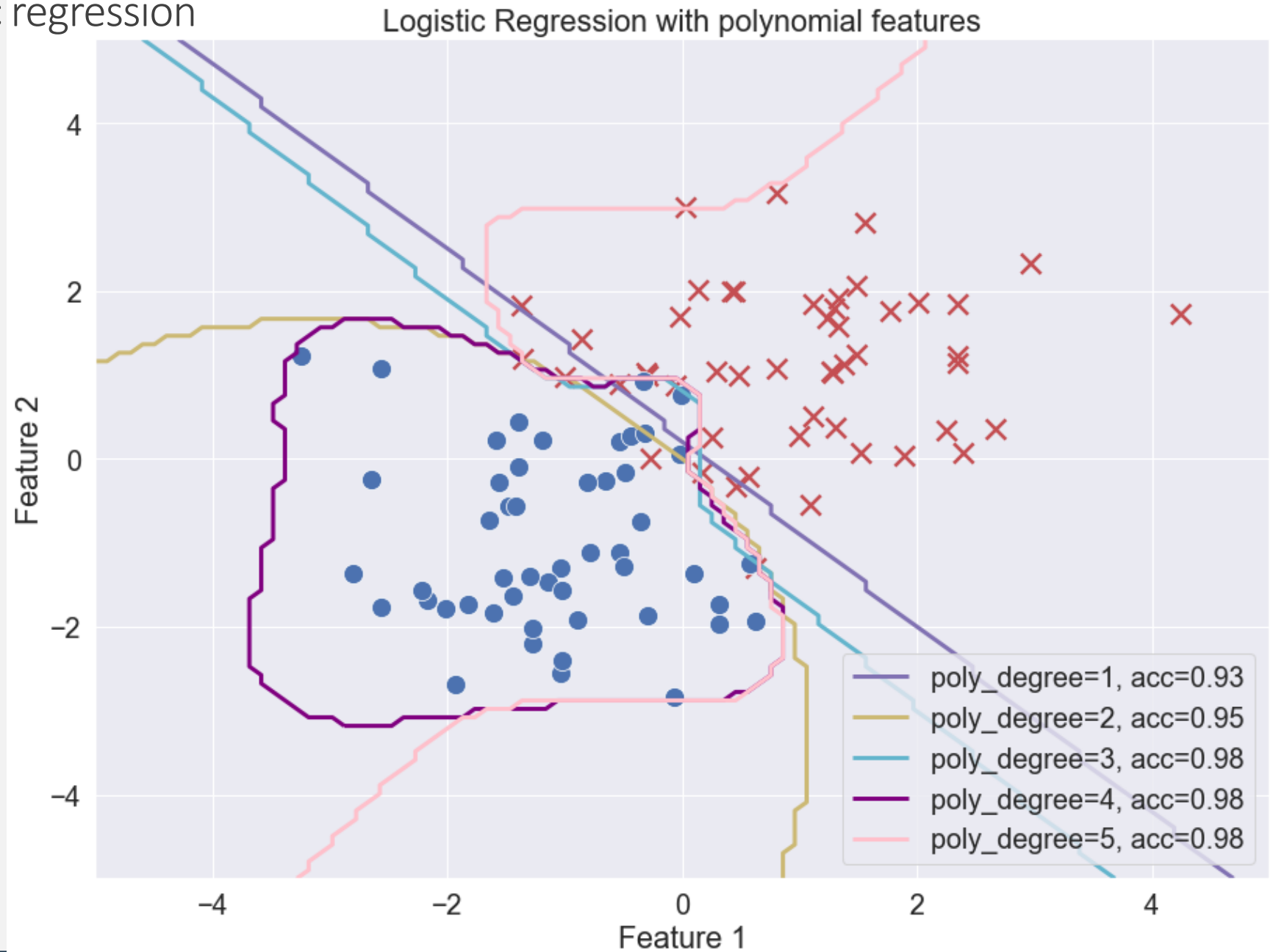


Terms with more than one feature
interacting are called interactions

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & x_{11}x_{12} & x_{11}^2 & x_{12}^2 & x_{11}^2x_{12} & x_{11}x_{12}^2 & x_{11}^3 & x_{12}^3 \\ 1 & x_{21} & x_{22} & x_{21}x_{22} & x_{21}^2 & x_{22}^2 & x_{21}^2x_{22} & x_{21}x_{22}^2 & x_{21}^3 & x_{22}^3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{m1} & x_{m2} & x_{m1}x_{m2} & x_{m1}^2 & x_{m2}^2 & x_{m1}^2x_{m2} & x_{m1}x_{m2}^2 & x_{m1}^3 & x_{m2}^3 \end{bmatrix}$$

Example of polynomial logistic regression

- We can convert our features to polynomial features then apply logistic regression.
- As a result, the decision boundary is non-linear.



Scikit-learn Polynomial Feature generator

```
class sklearn.preprocessing.PolynomialFeatures(degree=2, *,  
interaction_only=False, include_bias=True, order='C') \[source\]
```

```
from sklearn.preprocessing import PolynomialFeatures  
from sklearn.linear_model import LogisticRegression  
  
pf_generator = PolynomialFeatures(degree=4)  
  
X_train_poly = pf_generator.fit_transform(X_train)  
  
pr_model = LogisticRegression(fit_intercept=False)  
pr_model.fit(X_train_poly, y_train)  
  
X_test_poly = pf_generator.fit_transform(X_test)  
pr_model.predict(X_test_poly)
```

- [PolynomialFeatures](#) takes in the original feature matrix (without the 1 column) and generates polynomial feature matrix up to the specified degree. (see short guide [here](#))
- Can also use it as part of a pipeline.
- Note: we set `fit_intercept=False` since `PolynomialFeatures` will add 1-column by default (`include_bias=True`).

Multiclass Classification

Multiclass Classification [1]

- In a multiclass classification problem, the label outputs are K classes. That is, $y_i \in \{0, 1, 2, \dots, K - 1\}$. For example, for a 3-class task, $y_i \in \{0, 1, 2\}$.
- ML algorithms like Naive Bayes, decision trees, and k -nearest neighbors can handle multiclass tasks directly without modification.
- So far, our discussion on logistic regression was restricted to just binary classification.
- How can we modify logistic regression for multiclass classification?

Multiclass Classification [2]

- How can we modify logistic regression for multiclass classification?
- For **any** ML algorithm that can perform binary classification, we can modify the algorithm to perform K -class classification in two ways:
 1. **One vs All (OvA)**: builds K binary classification models, one for each class.
 2. **One vs One (OvO)**: builds $\frac{K(K-1)}{2}$ binary classification models, one for each *pair* of classes.
- Also, there is a specific formulation for multiclass logistic regression known as multinomial regression.

One vs. All (Also known as One vs Rest) [1]

- Given a training set $T = (X, Y) = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, where $y_i \in \{0, 1, 2, \dots, K - 1\}$. Consider a binary classification algorithm M_{algo} that outputs **probability scores**.
- **One vs All (OvA)**: builds K binary classification models, one for each class.

Training Procedure:

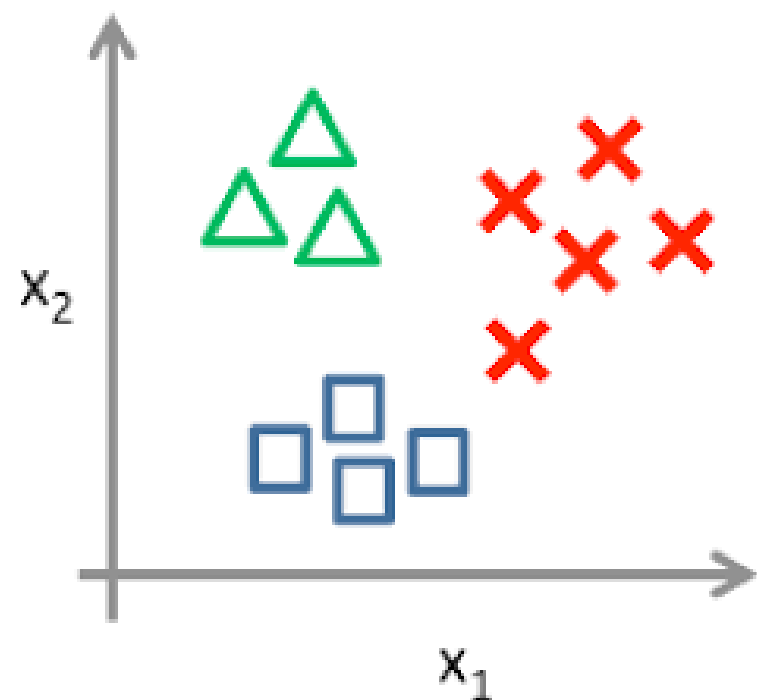
for c in $\{0, 1, \dots, K - 1\}$:

1. make a new training set $T^c = (X, Y^c)$ where $Y_i^c = 1$ if $Y_i = c$, otherwise $Y_i^c = 0$.
2. Train M_{algo} on $T^c = (X, Y^c)$. We will refer to this trained model as h_c .

Prediction Procedure:

Given a new instance x_{new} , then $y_{new} = \arg \max_{c \in \{0, 1, \dots, K - 1\}} h_c(x_{new})$

One-vs-all (one-vs-rest):



Class 1: **Green**
Class 2: **Blue**
Class 3: **Red**

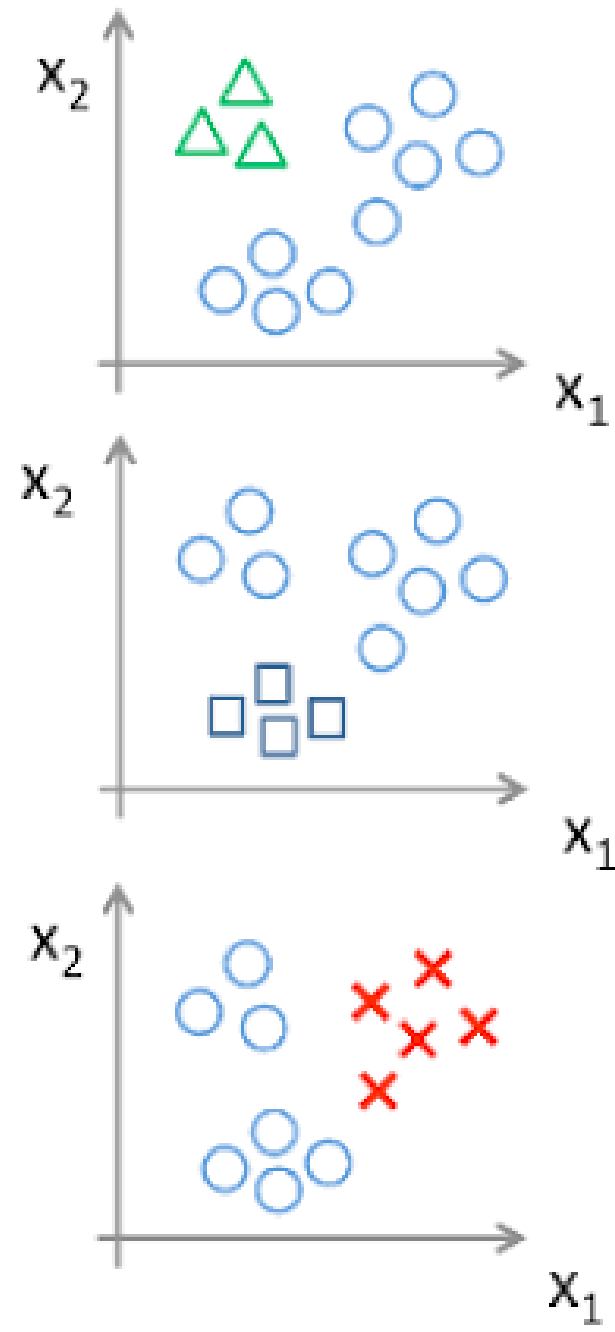
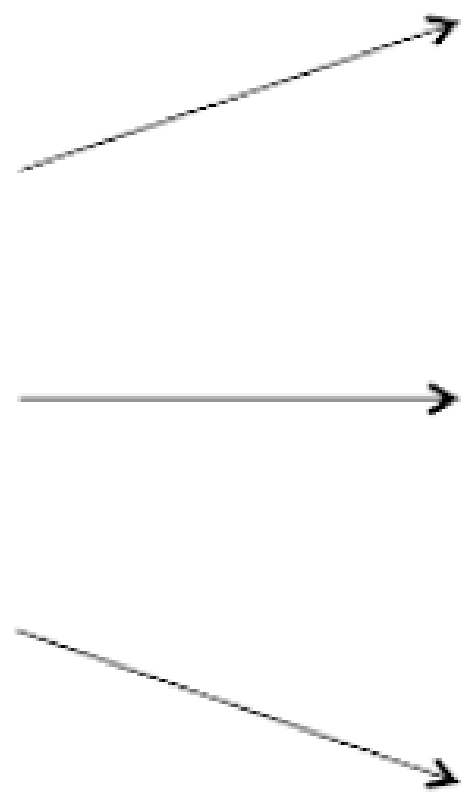


Image from: [link](#)

One vs. All (Also known as One vs Rest) [3]

Feature 1	Feature 2	Output Class
0.59	0.08	Red
0.10	0.12	Green
0.03	0.92	Red
0.47	0.23	Green
0.35	0.28	Blue
0.29	0.48	Blue

3-class dataset.
Split into 3 OvA datasets.
Model trained on each.

Red vs All

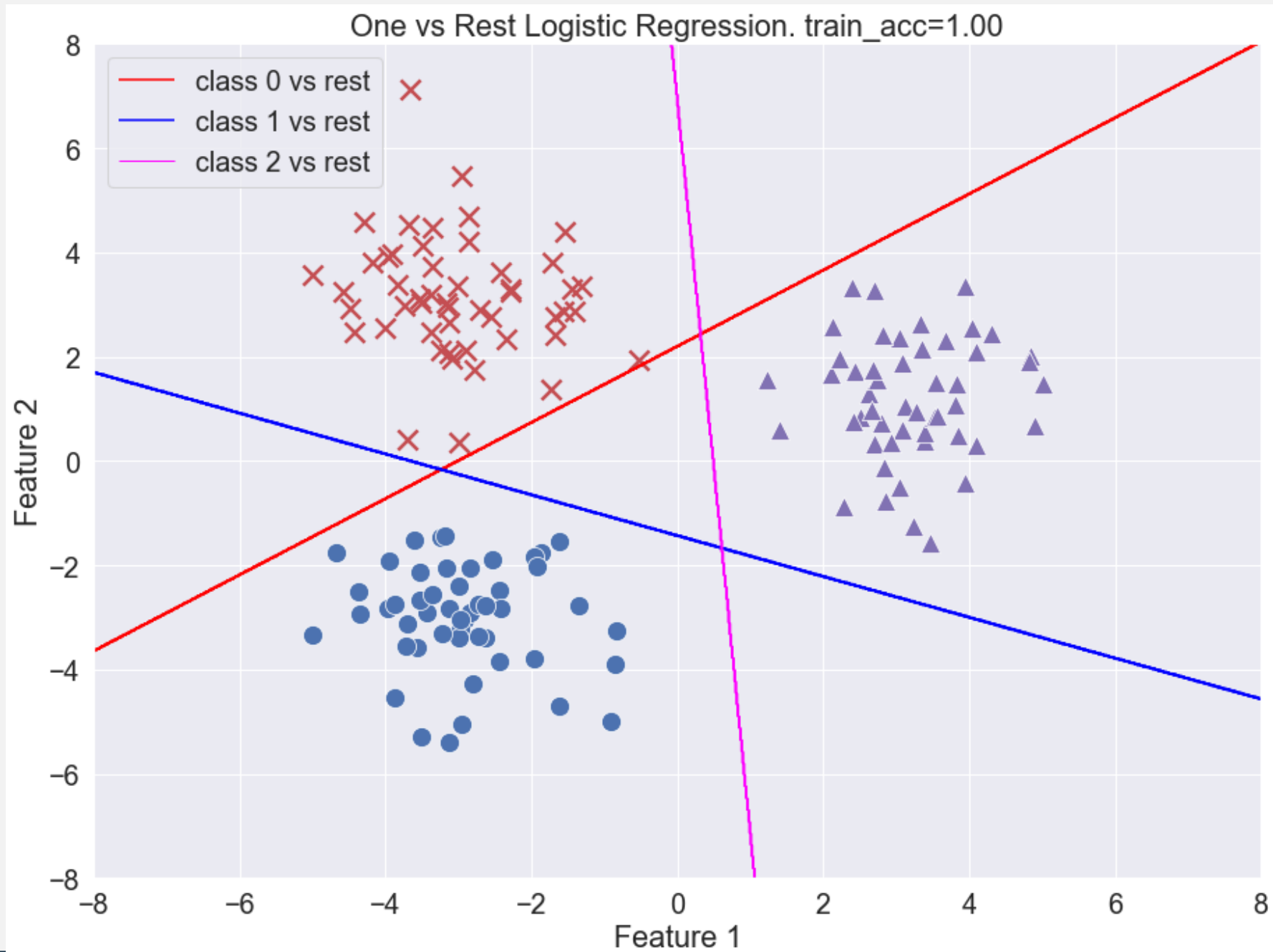
Feature 1	Feature 2	Output Class
0.59	0.08	1
0.10	0.12	0
0.03	0.92	1
0.47	0.23	0
0.35	0.28	0
0.29	0.48	0

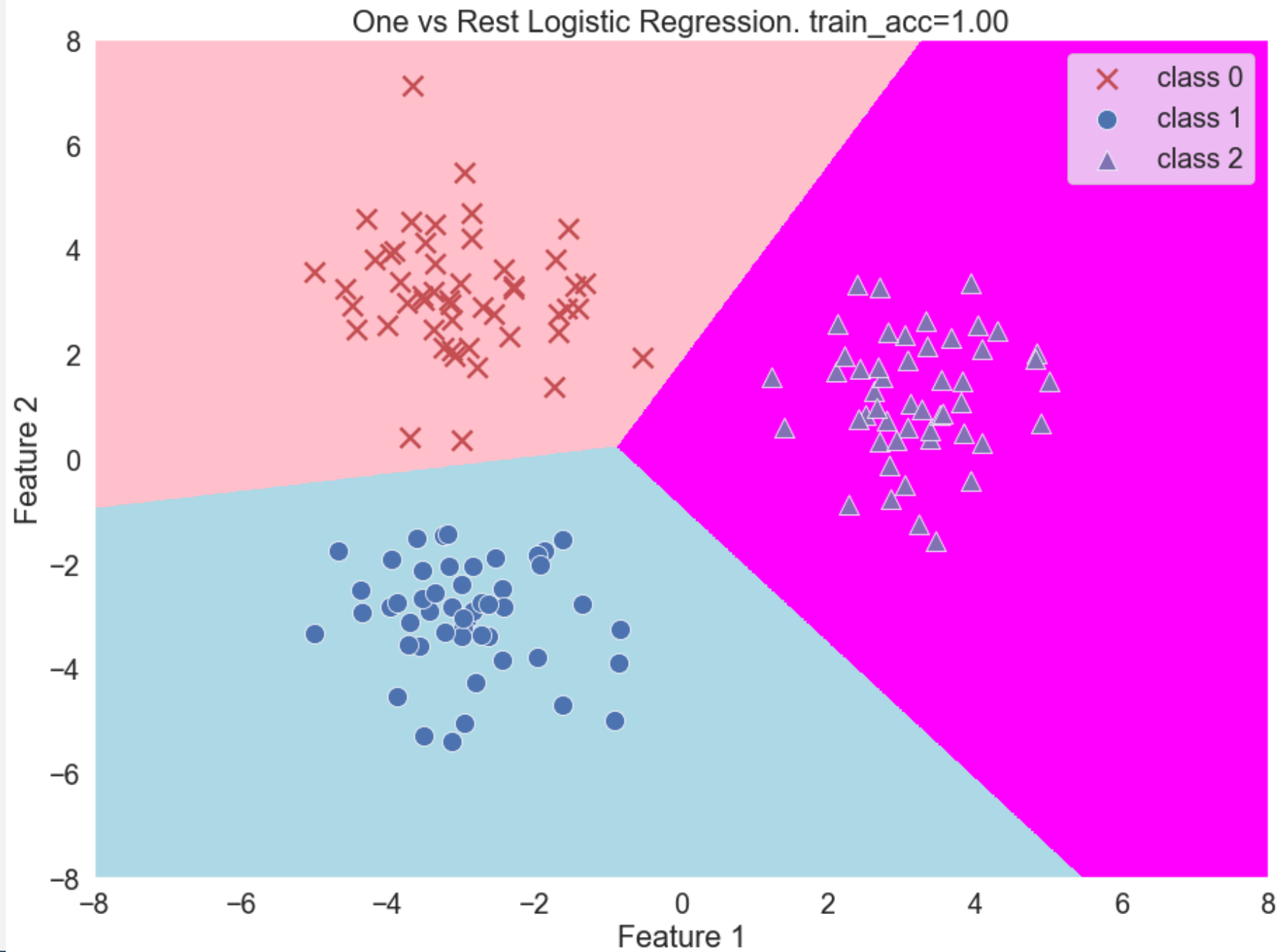
Green vs All

Feature 1	Feature 2	Output Class
0.59	0.08	0
0.10	0.12	1
0.03	0.92	0
0.47	0.23	1
0.35	0.28	0
0.29	0.48	0

Blue vs All

Feature 1	Feature 2	Output Class
0.59	0.08	0
0.10	0.12	0
0.03	0.92	0
0.47	0.23	0
0.35	0.28	1
0.29	0.48	1





One vs. One [1]

- Given a training set $T = (X, Y) = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, where $y_i \in \{0, 1, 2, \dots, K - 1\}$. Consider a binary classification algorithm M_{algo} that outputs **probability scores**.
- **One vs One (OvO)**: builds $\frac{K(K-1)}{2}$ binary classification models, one for each *pair* of classes.

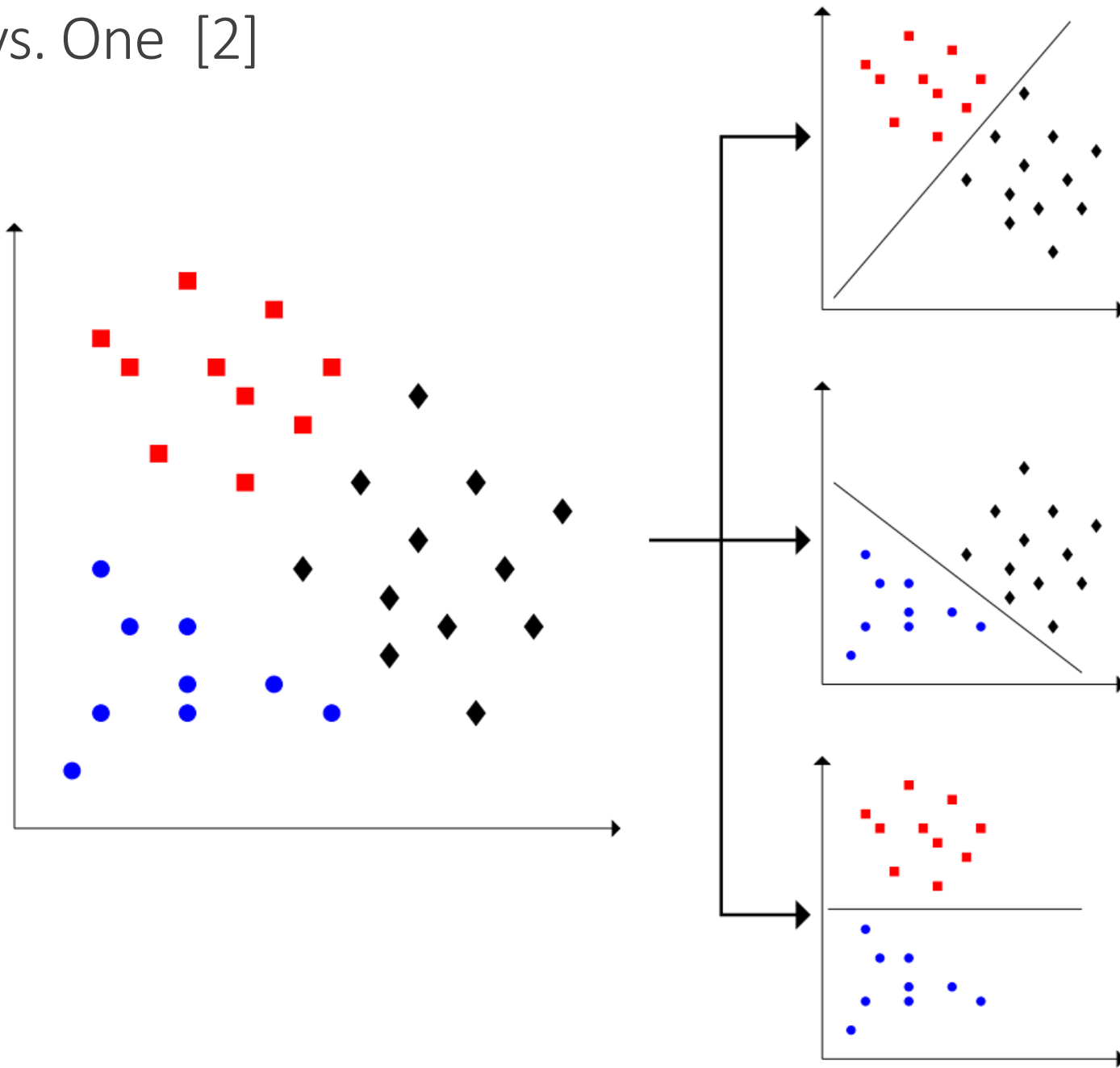
Training Procedure:

For each pair of classes, construct a training set where the labels are just those two classes, then train a model on this paired-training set. In total, $\frac{K(K-1)}{2}$ models.

Prediction Procedure:

Given a new instance x_{new} , compute prediction using the $\frac{K(K-1)}{2}$ models and take the majority vote (the class that had the most *wins*).

One vs. One [2]



3 classes, so $\frac{3*(3-1)}{2} = 3$ models.
red vs black
blue vs black
red vs blue

Image from: [link](#)

One vs. One [3]

Feature 1	Feature 2	Output Class
0.59	0.08	Red
0.10	0.12	Green
0.03	0.92	Red
0.47	0.23	Green
0.35	0.28	Blue
0.29	0.48	Blue

3-class dataset.
Split into $\frac{3*(3-1)}{2}$ OvO datasets.
Model trained on each.

Red vs Green

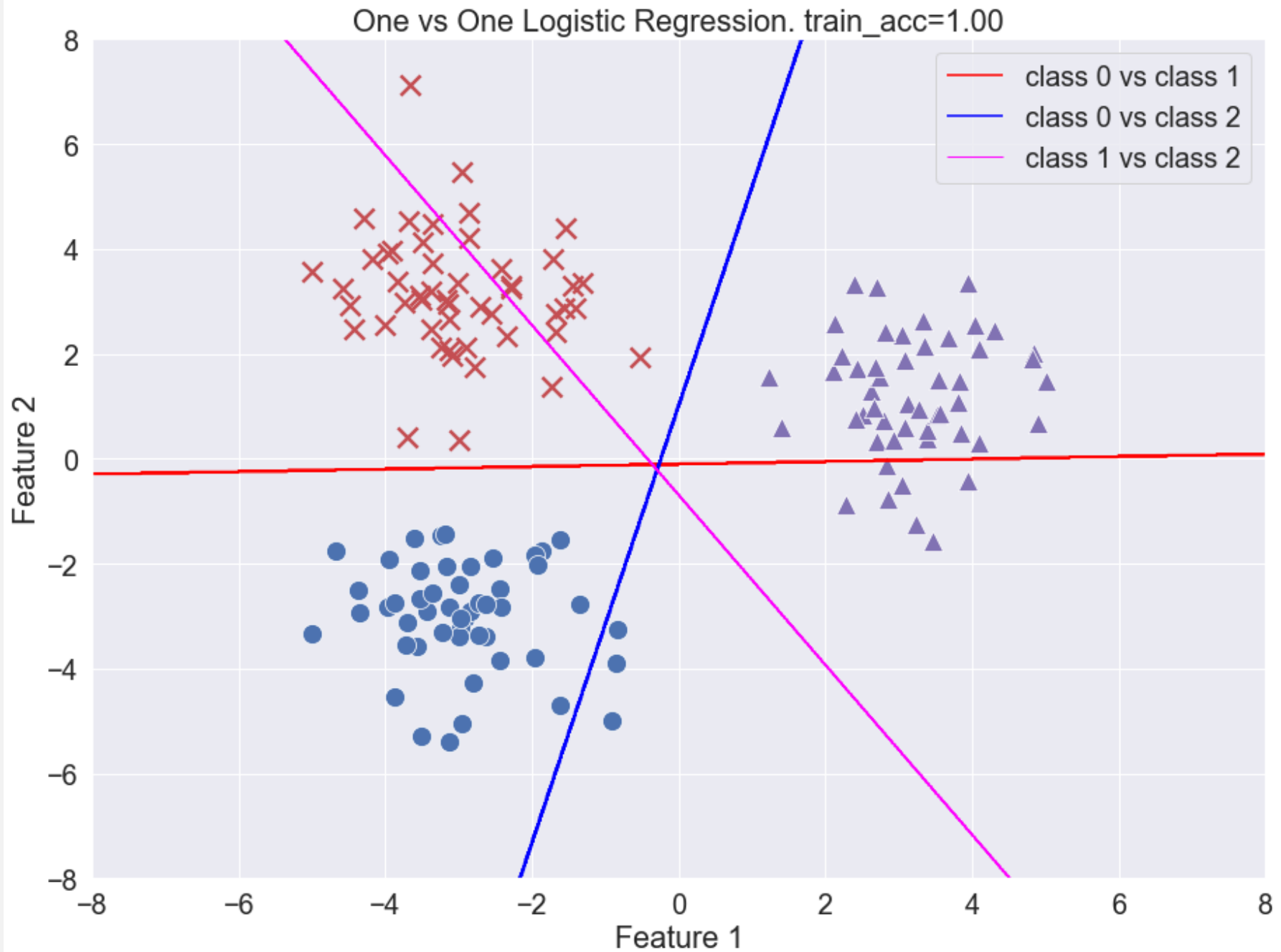
Feature 1	Feature 2	Output Class
0.59	0.08	1
0.10	0.12	0
0.03	0.92	1
0.47	0.23	0

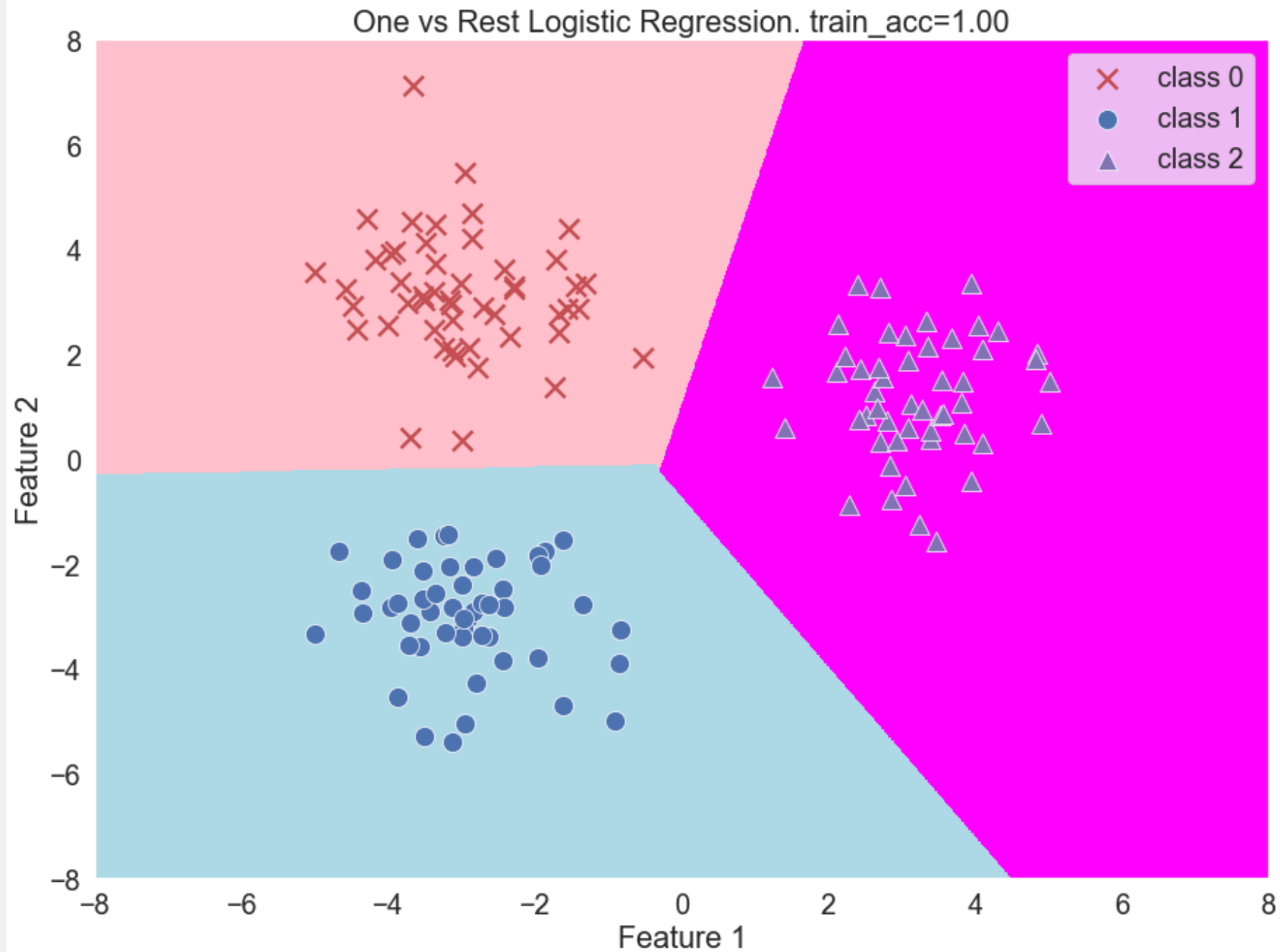
Red vs Blue

Feature 1	Feature 2	Output Class
0.59	0.08	1
0.03	0.92	1
0.35	0.28	0
0.29	0.48	0

Green vs Blue

Feature 1	Feature 2	Output Class
0.10	0.12	1
0.47	0.23	1
0.35	0.28	0
0.29	0.48	0





Multinomial Classification

Multinomial Classification [1]

- In a multiclass classification problem, the label outputs are K classes; i.e. $y_i \in \{0, 1, 2, \dots, K - 1\}$.
- Multinomial classification is a logistic regression formulation for the multiclass case.
- Each of the K classes are modelled as a probability where they sum to 1:

$$\begin{aligned} Prob(y_i = 0|x_i) &= p_0 \\ Prob(y_i = 1|x_i) &= p_1 \\ Prob(y_i = 2|x_i) &= p_2 \\ &\vdots \\ Prob(y_i = K|x_i) &= p_K \end{aligned} \quad \text{such that } \sum_{j=1}^K p_j = 1$$

Multinomial Classification [2]

$$Prob(y_i = 0|x_i) = p_0$$

$$Prob(y_i = 1|x_i) = p_1$$

$$Prob(y_i = 2|x_i) = p_2$$

$$\vdots$$

$$Prob(y_i = K|x_i) = p_K$$

such that $\sum_{j=1}^K p_j = 1$

- We approximate p_j via the following formulation:

$$Prob(y_i = c|x_i) = \frac{e^{x_i^T w^{(c)}}}{\sum_{j=1}^K e^{x_i^T w^{(j)}}}$$

Each class will have its own weight vector $w^{(c)}$. The denominator is to normalize the scores to be in range of (0, 1)

Multinomial Classification [3]

$$Prob(y_i = 0|x_i) = \frac{e^{x_i^T w^{(0)}}}{\sum_{j=1}^K e^{x_i^T w^{(j)}}}$$

$$Prob(y_i = 1|x_i) = \frac{e^{x_i^T w^{(1)}}}{\sum_{j=1}^K e^{x_i^T w^{(j)}}}$$

$\vdots$

$$Prob(y_i = K|x_i) = \frac{e^{x_i^T w^{(K)}}}{\sum_{j=1}^K e^{x_i^T w^{(j)}}}$$

Note that $\sum_{j=1}^K Prob(y_i = j|x_i) = 1$

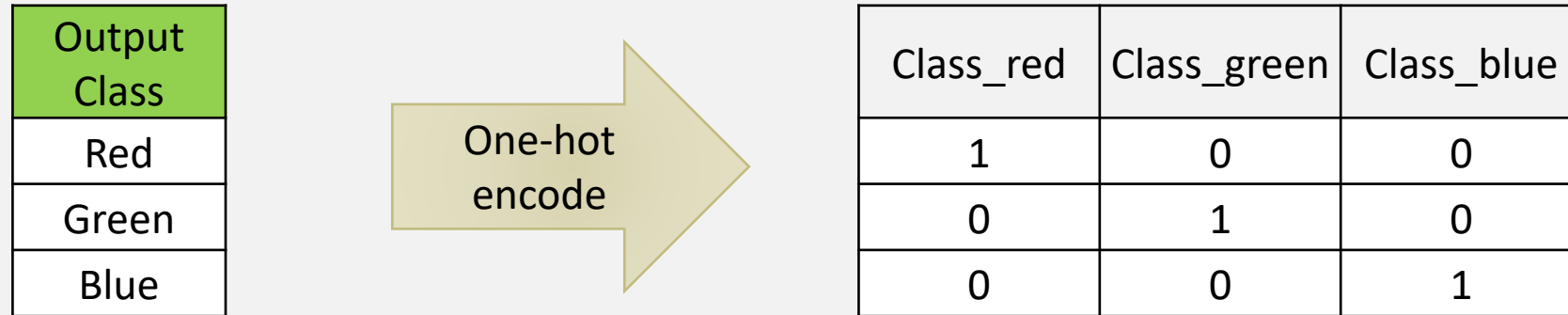
Each class will have its own weight vector $w^{(c)}$. The denominator is to normalize the scores to be in range of (0, 1)

Notice that large values of $x_i^T w^{(c)}$ correspond to larger probability scores.

This treatment is also called softmax.

Multinomial Classification [4]

- The output labels are one-hot encoded to simplify the loss formula and corresponding gradient.



- Thus, the training set $T = (X, Y) = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, where $y_i \in \{0, 1, 2, \dots, K - 1\}$. After one-hot encoding y_i , they become vectors.
- We use a similar loss to the binary logloss but adapted for multiclass:

$$w^{(0)}, w^{(1)}, \dots, w^{(K)} = \arg \min - \sum_{i=1}^m y_i^T \log (Prob(y_i | x_i))$$

Note that we are doing vector dot product

Multinomial Classification [6]

$$w^{(0)}, w^{(1)}, \dots, w^{(K)} = \arg \min - \sum_{i=1}^m y_i^T \log (Prob(y_i|x_i))$$

$$\text{where } Prob(y_i|x_i) = \begin{pmatrix} \frac{e^{x_i^T w^{(0)}}}{\sum_{j=1}^K e^{x_i^T w^{(j)}}} \\ \frac{e^{x_i^T w^{(1)}}}{\sum_{j=1}^K e^{x_i^T w^{(j)}}} \\ \vdots \\ \frac{e^{x_i^T w^{(K)}}}{\sum_{j=1}^K e^{x_i^T w^{(j)}}} \end{pmatrix}$$

Multinomial Classification [7]

$$w^{(0)}, w^{(1)}, \dots, w^{(K)} = \arg \min - \sum_{i=1}^m y_i^T \log (Prob(y_i|x_i))$$

- To find the minimum solution, we use gradient descent. This multiclass logloss has a gradient, because the softmax has a gradient.
- Each update will update ALL the K weight vectors.

$$\hat{w}^{(j)} := \hat{w}^{(j)} - \alpha \frac{\partial \logloss(\hat{w}^{(j)})}{\partial w^{(j)}}$$

Multinomial Classification [8]

- After training, we can predict the probability of each class on a new instance x_{new} :

$$Prob(y_{new}|x_{new}) = \begin{pmatrix} Prob(y_{new} = 0|x_{new}) \\ Prob(y_{new} = 1|x_{new}) \\ \vdots \\ Prob(y_{new} = K|x_{new}) \end{pmatrix} = \begin{pmatrix} \frac{e^{x_{new}^T w^{(0)}}}{\sum_{j=1}^K e^{x_{new}^T w^{(j)}}} \\ \frac{e^{x_{new}^T w^{(1)}}}{\sum_{j=1}^K e^{x_{new}^T w^{(j)}}} \\ \vdots \\ \frac{e^{x_{new}^T w^{(K)}}}{\sum_{j=1}^K e^{x_{new}^T w^{(j)}}} \end{pmatrix}$$

- We can force a class label by taking the class with the highest probability.

